

C.14 ALGORITMI AVANSATI FOLOSITI IN SECURITATEA CIBERNETICA

PAUL A. GAGNIUC



Academia Tehnică Militară „Ferdinand I”

PRINCIPALELE PĂRȚI ALE PREZENTĂRII

C.14 Algoritmi avansati folositi in securitatea cibernetica:

- C.14.1 TIPURI DE SALTURI ȘI IMPLICAȚII PENTRU DETECTAREA MALWARE
- C.14.2 SEMNĂTURI MALWARE
- C.14.3 MOTOARE ANTIVIRUS
- C.14.4 ALGORITMI FOLOSITI IN SECURITATEA CIBERNETICA

De citit:

Antivirus Engines

From Methods to Innovations, Design, and Applications



Paul A. Gagniuc

SYNGRESS

<https://github.com/gagniuc/Antivirus-Engines>



National University of Science and
Technology Politehnica Bucharest

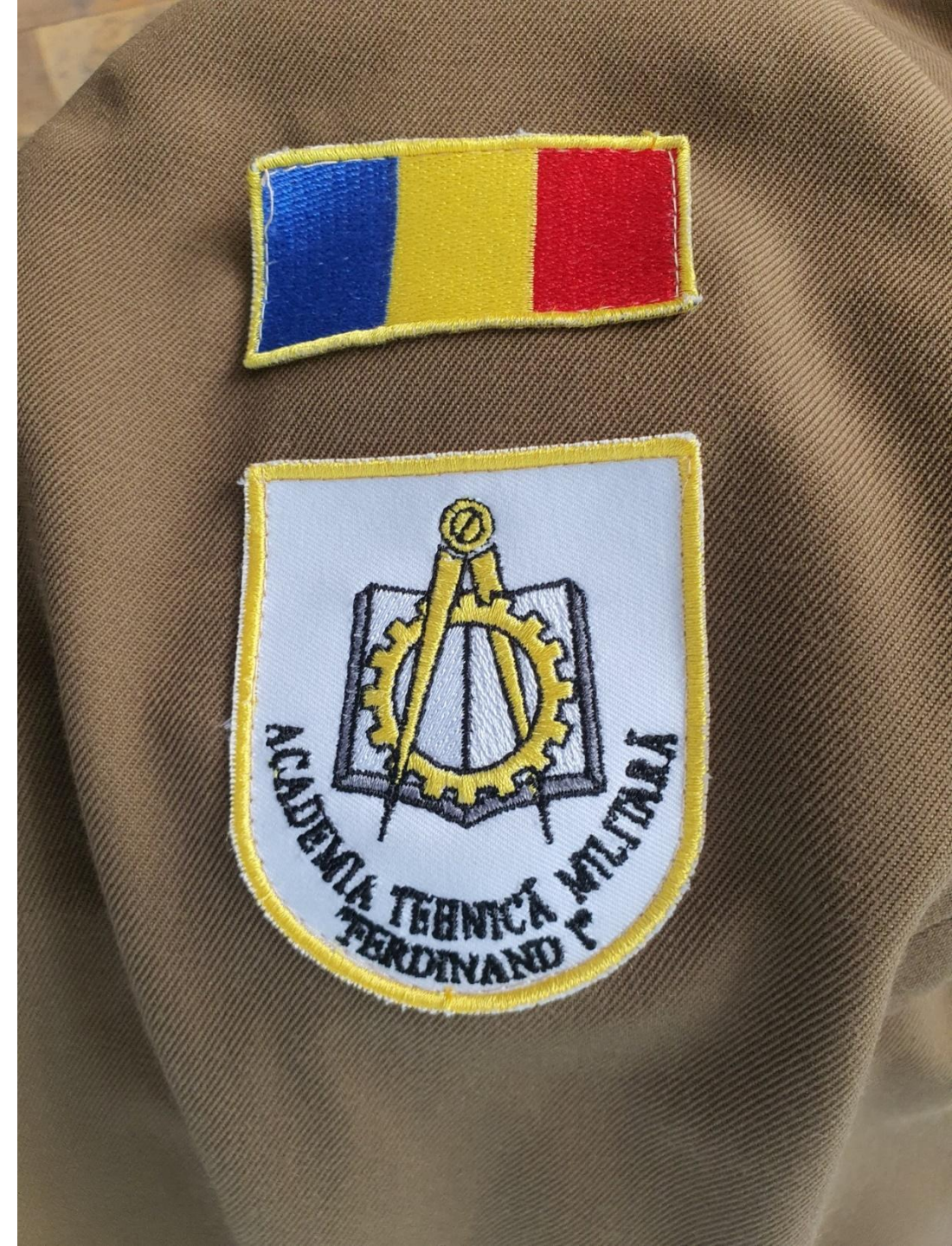


Military Technical Academy "Ferdinand I"



De reținut:

- Antivirus ! Nu există unul „cel mai bun”, deoarece intervin parametri multipli și versiuni diferite.
- Antivirusurile se împart în două categorii: abordări îndrăznețe și abordări sigure (bazate pe prevenție).
- Ce ne dorim de la un antivirus? Consum redus de resurse; testele de tip benchmark sunt adesea irelevante.
- Tipuri de malware? Clasic și emergent.
- Metode de detecție: hashing, analiză hexadecimală, euristică.
- Pool-ul de semnături commune prezintă pericolul de overfitting.
- Pool-ul de exploitudini a scăzut progresiv datorită restructurării sistemelor de operare.
- Se poate face update sistemului de operare, dar nu și utilizatorului.



Securitate cibernetică de la cercetare la profesor

Entropie >

Cercetare



Industrie

< Entropie

Anomalii civilizaționale

Inginerie inversă de cod

Ce plănuiește?

Comportament

Ce face?

Ce este fișierul suspect?
Ce face fișierul suspect?
Ce are particular fișierul suspect?

Laborator

Antivirus

Cum arată?
Ce caut?

Acasa/Birou

Integritatea civilizației

C.14.1

TIPURI DE SALTURI ȘI IMPLICATII PENTRU DETECTAREA MALWARE



INTRUCȚIUNEA E9 ESTE ABSOLUT NORMALĂ ÎN COD LEGITIM! CUM FOLOSEȘTE MALWARE-UL E9?

1. Hooking (deturnarea funcțiilor)

Malware-ul poate înlocui începutul unei funcții legitime (ex: `CreateProcess`, `OpenFile`) cu:

`E9 <offset> ; jmp` către codul său propriu

Astfel, când programul apelează funcția, sare în codul atacatorului

2. Redirecționarea fluxului de execuție

În viruși și troieni injectați în procese, apare des: `E9` în secțiuni neașteptate (ex: `.text` modificat, sau `.rdata` corupt)

Redirecționează către cod ofuscat, loader, shellcode etc.

3. Obfuscare sau polimorfism

Malware-ul își „sparge” codul în bucăți și face salturi între ele cu `jmp E9`

Asta îngreunează analiza statică (ex: în `IDA` sau `x64dbg`)

`E9` este opcode-ul pentru un salt lung

`E9 xx xx xx xx` → `jmp <relativ offset>`

Cum ne dăm seama dacă un `E9` este suspect?

- Se află la începutul unei funcții standard? (ex: în `kernel32.dll`) → suspect
- Sare în afara modulului? → suspect
- Nu are referințe legitime (`xref`)? → suspect
- Este într-o secțiune marcată ca `rdata` sau `idata`? → foarte suspect

CUM FOLOSEȘTE MALWARE-UL E9?

HOOKING

```
format PE64 console
entry start

include 'win64a.inc'

section '.text' code readable executable

start:
    call hooked_function

    invoke ExitProcess, 0

hooked_function:
    jmp malicious_code    ; E9 - simulate hook

legit_code:
    invoke MessageBoxA, 0, msg_legit, title, 0
    ret

malicious_code:
    invoke MessageBoxA, 0, msg_mal, title, 0
    ret

section '.data' data readable writeable

msg_legit db "Funcție legitimă", 0
msg_mal db "Funcție redirectionată (hooked)", 0
title db "Hooking E9", 0
```

Ce face acest exemplu?

Acest cod FASM simulează un **hook** aplicat unei funcții legitime.

- Funcția `hooked_function` ar trebui să apeleze `legit_code`
- Dar este înlocuită cu un **JMP (E9)** către **`malicious_code`**
- Astfel, controlul programului este deturnat spre un cod alternativ

Aceasta este o tehnică frecvent folosită de malware pentru a intercepta funcții din sistem sau din aplicații legitime.

● **Apelul original este păstrat, dar deturnat!**

CUM FOLOSEȘTE MALWARE-UL E9?

REDIRECTION

```
format PE64 console
entry start

include 'win64a.inc'

section '.text' code readable executable

start:
    jmp injected_code ; E9 în .text care sare în cod "injectat"

normal_execution:
    invoke MessageBoxA, 0, msg_normal, title, 0
    invoke ExitProcess, 0

injected_code:
    invoke MessageBoxA, 0, msg_injected, title, 0
    jmp normal_execution

section '.data' data readable writeable

msg_normal db "Execuție normală", 0
msg_injected db "Cod injectat (simulat)", 0
title db "Redirecționare E9", 0
```

Ce ilustrează acest exemplu?

Acest cod simulează un caz de **redirecționare a execuției** folosind instrucțiunea jmp (E9), exact cum fac virușii și troienii injectați.

- Execuția ar trebui să înceapă normal
- Însă un jmp de la începutul programului duce în cod „injectat”
- Acesta afișează un mesaj, apoi revine la fluxul original

Este o tehnică tipică de malware pentru a insera shellcode sau loaderi în secțiuni modificate precum **.text** sau **.rdata**.

CUM FOLOSEȘTE MALWARE-UL E9?



OBFUSCATION

```
format PE64 console
entry start

include 'win64a.inc'

section '.text' code readable executable

start:
    jmp part1

part3:
    invoke MessageBoxA, 0, msg_final, title, 0
    invoke ExitProcess, 0

part1:
    xor rax, rax
    jmp part2

part2:
    add rax, 1
    jmp part3

section '.data' data readable writeable

msg_final db "Cod fragmentat prin E9", 0
title db "Obfuscare", 0
```

Ce face acest exemplu?

Acest cod demonstrează o tehnică simplă de **obfuscare** folosită de malware.

- Codul este „spart” în bucăți (part1, part2, part3)
- Execuția sare între ele folosind jmp (E9)
- Deși logic ar putea fi scris secvențial, e fragmentat intenționat

Această tehnică este folosită pentru a **îngreuna analiza statică** în decompilatoare precum IDA sau Ghidra, unde codul pare întrerupt și greu de urmărit.



MALWARE OPCODE SIGNATURES:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Paul> & C:\Users\Paul\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Paul/Desktop/New folder/scan.py"
▲ Găsit cod suspect în: c:\Users\Paul\Desktop\New folder\exemplu.exe
- Regula: NOP_Sled
  ↳ Match brut: $sled
PS C:\Users\Paul>
```

Malware_Opcode_Signatures.yar

Semnături comune de malware folosind wildcards (??):

```
rule Hooking_JMP_E9
{
    strings:
        $hook = { E9 ?? ?? ?? ?? }
    condition:
        $hook
}

rule NOP_Sled
{
    strings:
        $sled = { 90 90 90 90 90 }
    condition:
        $sled
}

rule Jump_Obfuscation
{
    strings:
        $jmp = { EB ?? EB ?? EB ?? }
    condition:
        $jmp
}

rule Shellcode_Allocator
{
    strings:
        $alloc = { 6A 00 68 ?? ?? ?? ?? E8 }
    condition:
        $alloc
}

rule Reflective_Jump
{
    strings:
        $vjmp = { FF 15 ?? ?? ?? ?? E9 }
    condition:
        $vjmp
}
```

Acestea sunt **semnături binare** care detectează **pattern-uri de instrucțiuni (opcode-uri)** caracteristice pentru comportamente folosite frecvent de malware.

Aceste semnături pot fi:

- mai generale (cu mai mulți ?? → tolerant la variații)
- mai specifice (ex: incluzând și adrese, registre)
- bazate pe expresii regulate în stringuri
- combinate cu condiții logice complexe

1. Hooking – E9 ?? ?? ?? ??
2. NOP sled – 90 90 90 90 90
3. Obfuscation jump – EB ?? EB ?? EB ??
4. Shellcode alloc – 6A 00 68 ?? ?? ?? ?? E8
5. Reflective loader (VirtualAlloc + JMP) – FF 15 ?? ?? ?? ?? E9

Aceste semnături sunt euristice – ele caută modele comune în malware, nu secvențe exacte de fișiere.

```
import yara
import os
```

pip install yara-python

```
# Cale relativă:
current_dir = os.path.dirname(os.path.abspath(__file__))
rules = yara.compile(filepath=os.path.join(current_dir,
"Malware_Opcode_Signatures.yar"))
target_file = os.path.join(current_dir, "exemplu.exe")

matches = rules.match(target_file)

if matches:
    print(f"Găsit cod suspect în: {target_file}")
    for match in matches:
        print(f"- Regula: {match.rule}")
        for s in match.strings:
            print(f"  ↳ Match brut: {repr(s)}")
else:
    print(f"Nimic suspect în: {target_file}")
```

C.10.2

SEMNĂTURI MALWARE



Metode de Semnături de Malware

```
graph LR; A[Metode de Semnături de Malware] --- B[Semnături Statice]; A --- C[Semnături Dinamice]; A --- D[Semnături Bazate pe String-uri]; A --- E[Semnături Heuristice]; A --- F[Semnături de Polimorfism]; A --- G[Semnături de Anomalie];
```

Semnături Statice

Descriere: Identifică malware-ul prin compararea hash-urilor fișierelor suspecte cu o bază de date de hash-uri ale malware-urilor cunoscute.

Semnături Dinamice

Descriere: Se bazează pe analiza comportamentului la executarea într-un mediu controlat (sandbox) pentru a genera semnături bazate pe acțiunile detectate.

Semnături Bazate pe String-uri

Descriere: Caută secvențe specifice de text sau cod într-un fișier care sunt caracteristice malware-ului.

Semnături Heuristice

Descriere: Folosește reguli sau algoritmi pentru a identifica tipare sau caracteristici ale codului care sugerează un comportament malițios.

Semnături de Polimorfism

Descriere: Detectează variantele de malware care folosesc tehnici de polimorfism pentru a-și modifica codul la fiecare infecție, îngreunând detectarea.

Semnături de Anomalie

Descriere: Identifică activitățile suspecte sau neobișnuite bazându-se pe un profil de normalitate stabilit anterior.

Metode de Semnături Dinamice

```
graph LR; A[Metode de Semnături Dinamice] --- B[Analiza Comportamentului]; A --- C[Analiza Traficului de Rețea]; A --- D[Analiza Modificărilor de Sistem]; A --- E[Analiza Proceselor]; A --- F[Analiza Memoriei]; B --- B_desc[Descriere: Observă comportamentul aplicațiilor în timp real sau într-un mediu sandbox pentru a identifica acțiuni suspecte.]; C --- C_desc[Descriere: Monitorizează traficul de rețea pentru a detecta comunicații neobișnuite sau malițioase.]; D --- D_desc[Descriere: Verifică modificările în fișiere, registrii sistemului sau setări, indicând potențiala activitate malițioasă.]; E --- E_desc[Descriere: Identifică procesele neobișnuite sau suspecte care rulează pe sistem, care ar putea indica prezența malware-ului.]; F --- F_desc[Descriere: Scanează memoria sistemului pentru a găsi mostre de cod malițios care se execută sau rezidă în memorie.];
```

Analiza Comportamentului

Descriere: Observă comportamentul aplicațiilor în timp real sau într-un mediu sandbox pentru a identifica acțiuni suspecte.

Analiza Traficului de Rețea

Descriere: Monitorizează traficul de rețea pentru a detecta comunicații neobișnuite sau malițioase.

Analiza Modificărilor de Sistem

Descriere: Verifică modificările în fișiere, registrii sistemului sau setări, indicând potențiala activitate malițioasă.

Analiza Proceselor

Descriere: Identifică procesele neobișnuite sau suspecte care rulează pe sistem, care ar putea indica prezența malware-ului.

Analiza Memoriei

Descriere: Scanează memoria sistemului pentru a găsi mostre de cod malițios care se execută sau rezidă în memorie.

Metode de Semnături Heuristice

```
graph LR; A[Metode de Semnături Heuristice] --- B[Analiza Statică]; A --- C[Analiza Dinamică]; A --- D[Detectarea Anomaliilor]; A --- E[Algoritmi de Clasificare]; A --- F[Analiza Entropiei]; A --- G[Semnături Bazate pe Grafuri]; A --- H[Învățarea Profundă];
```

Analiza Statică

Descriere: Examinarea codului fără a-l executa, identificând tipare sau elemente suspecte prin scanarea statică a fișierelor.

Analiza Dinamică

Descriere: Evaluarea comportamentului codului în timpul execuției într-un mediu virtual pentru a identifica acțiuni potențial malițioase.

Detectarea Anomaliilor

Descriere: Utilizează modele statistice pentru a identifica deviații de la comportamentul normal al aplicațiilor, semnalând posibilele amenințări.

Algoritmi de Clasificare

Descriere: Folosirea algoritmilor de învățare automată pentru a clasifica aplicațiile ca fiind malițioase sau benigne, bazându-se pe caracteristicile extrase din analiza statică și dinamică.

Analiza Entropiei

Descriere: Măsoară gradul de ordine sau dezordine într-un fișier pentru a detecta compresia sau criptarea, indicând posibile ascunzișuri de cod malițios.

Semnături Bazate pe Grafuri

Descriere: Compararea structurilor de grafuri generate din codul programelor cu grafuri asociate cu malware cunoscut, pentru a identifica similarități.

Învățarea Profundă

Descriere: Utilizarea rețelelor neuronale profunde pentru a analiza și învăța din seturi mari de date, identificând tipare complexe care indică malware.

Metode de Semnături Statice

```
graph LR; A[Metode de Semnături Statice] --- B[Compararea Hash-ului]; A --- C[Detectarea String-urilor]; A --- D[Analiza Pattern-urilor de Cod]; A --- E[Verificarea Semnăturilor Digitale]; A --- F[Analiza Metadata-ului];
```

Compararea Hash-ului

Descriere: Compară hash-urile fișierelor cu o bază de date de hash-uri ale malware-urilor cunoscute.

Detectarea String-urilor

Descriere: Caută secvențe specifice de caractere sau coduri în fișiere care sunt asociate frecvent cu malware.

Analiza Pattern-urilor de Cod

Descriere: Identifică pattern-uri de cod specifice care sunt caracteristice malware-ului, utilizând o combinație de tehnici de analiză statică.

Verificarea Semnăturilor Digitale

Descriere: Verifică semnăturile digitale ale fișierelor pentru a detecta modificări neautorizate sau fișiere care sunt semnate digital de entități suspecte.

Analiza Metadata-ului

Descriere: Examinează metadatele fișierului, cum ar fi data creării sau modificării, pentru a identifica activități suspecte.

C.10.3

MOTOARE ANTIVIRUS



Elemente Principale ale unui Motor Antivirus

```
graph LR; A[Elemente Principale ale unui Motor Antivirus] --- B[Scannerul de Semnături]; A --- C[Heuristica/Analiza Comportamentală]; A --- D[Motorul de Sandboxing]; A --- E[Sistemul de Prevenire a Intruziunilor (IPS)]; A --- F[Sistemul de Actualizare]; A --- G[Interfața cu Utilizatorul]; A --- H[Jurnalizarea și Raportarea]; A --- I[Quarantina];
```

Scannerul de Semnături

Descriere: Identifică malware-ul comparând cu o bază de date de semnături de malware cunoscute.

Heuristica/Analiza Comportamentală

Descriere: Detectează malware nou sau modificat analizând comportamentul sau caracteristicile fișierelor.

Motorul de Sandboxing

Descriere: Testează executabilele suspecte într-un mediu virtual sigur pentru a evalua comportamentul lor.

Sistemul de Prevenire a Intruziunilor (IPS)

Descriere: Monitorizează rețeaua și sistemul pentru activități suspecte, blocând atacurile cunoscute.

Sistemul de Actualizare

Descriere: Menține software-ul antivirus actualizat cu cele mai recente semnături și algoritmi de detectare.

Interfața cu Utilizatorul

Descriere: Permite utilizatorilor să configureze setările antivirus, să vizualizeze rapoarte și să inițieze scanări manuale.

Jurnalizarea și Raportarea

Descriere: Înregistrează evenimentele de securitate și oferă rapoarte detaliate despre detectările și acțiunile întreprinse.

Quarantina

Descriere: Izolează fișierele suspecte sau infectate pentru a preveni răspândirea malware-ului.

Metode de Detectie Antivirus

```
graph LR; A[Metode de Detectie Antivirus] --- B[Semnături de Virus]; A --- C[Analiza Euristică]; A --- D[Sandboxing]; A --- E[Monitorizarea Comportamentului]; A --- F[Integritatea Fișierelor]; A --- G[Analiza Rețelei]; A --- H[Inteligența Artificială și Învățarea Automată];
```

Semnături de Virus

Descriere: Compararea fișierelor cu o bază de date de semnături de malware cunoscute.

Analiza Euristică

Descriere: Identificarea comportamentelor suspecte sau a caracteristicilor asociate cu malware, bazându-se pe reguli sau modele.

Sandboxing

Descriere: Executarea programelor într-un mediu virtual izolat pentru a observa comportamentul acestora fără a risca sistemul.

Monitorizarea Comportamentului

Descriere: Supravegherea activităților sistemului în timp real pentru a detecta acțiuni neobișnuite sau periculoase.

Integritatea Fișierelor

Descriere: Verificarea modificărilor neautorizate ale fișierelor esențiale ale sistemului.

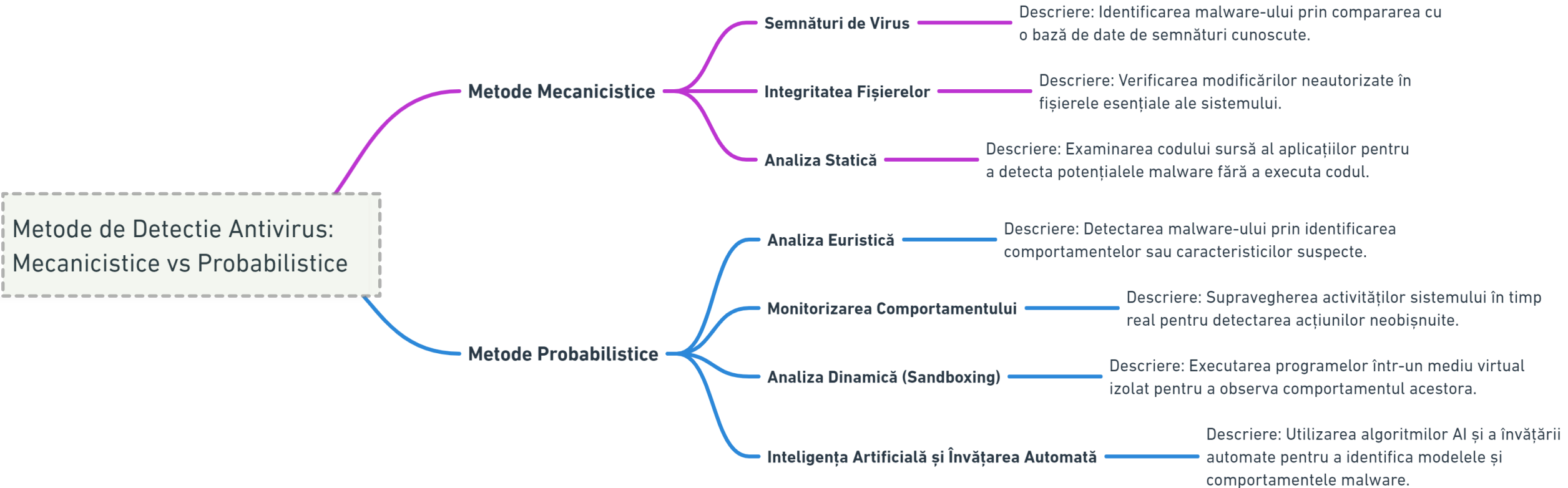
Analiza Rețelei

Descriere: Monitorizarea traficului de rețea pentru a identifica modele suspecte sau malware care comunică cu servere de comandă și control.

Inteligența Artificială și Învățarea Automată

Descriere: Utilizarea algoritmilor AI și a învățării automate pentru a identifica noi tipuri de malware pe baza unor modele complexe și a comportamentului.

Noțiuni elementare: detecție mecanicistă vs detecție probabilistică.



Metode de Dezinfecție ale Malware

```
graph LR; A[Metode de Dezinfecție ale Malware] --- B[Scanare și Îndepărtare]; A --- C[Restaurarea Sistemului]; A --- D[Reinstalarea Sistemului de Operare]; A --- E[Update și Patching]; A --- F[Curățarea Registrelor]; A --- G[Izolarea și Quarantina]; A --- H[Schimbarea Parolelor];
```

Scanare și Îndepărtare

Descriere: Utilizarea unui software antivirus pentru a scana sistemul, detecta și îndepărta malware-ul.

Restaurarea Sistemului

Descriere: Folosirea punctelor de restaurare ale sistemului pentru a reveni la o configurație anterioară și sănătoasă.

Reinstalarea Sistemului de Operare

Descriere: În cazurile severe, poate fi necesară reinstalarea completă a sistemului de operare pentru a elimina toate urmele de malware.

Update și Patching

Descriere: Actualizarea sistemului de operare și a software-ului cu cele mai recente patch-uri de securitate pentru a remedia vulnerabilitățile exploatare de malware.

Curățarea Registrelor

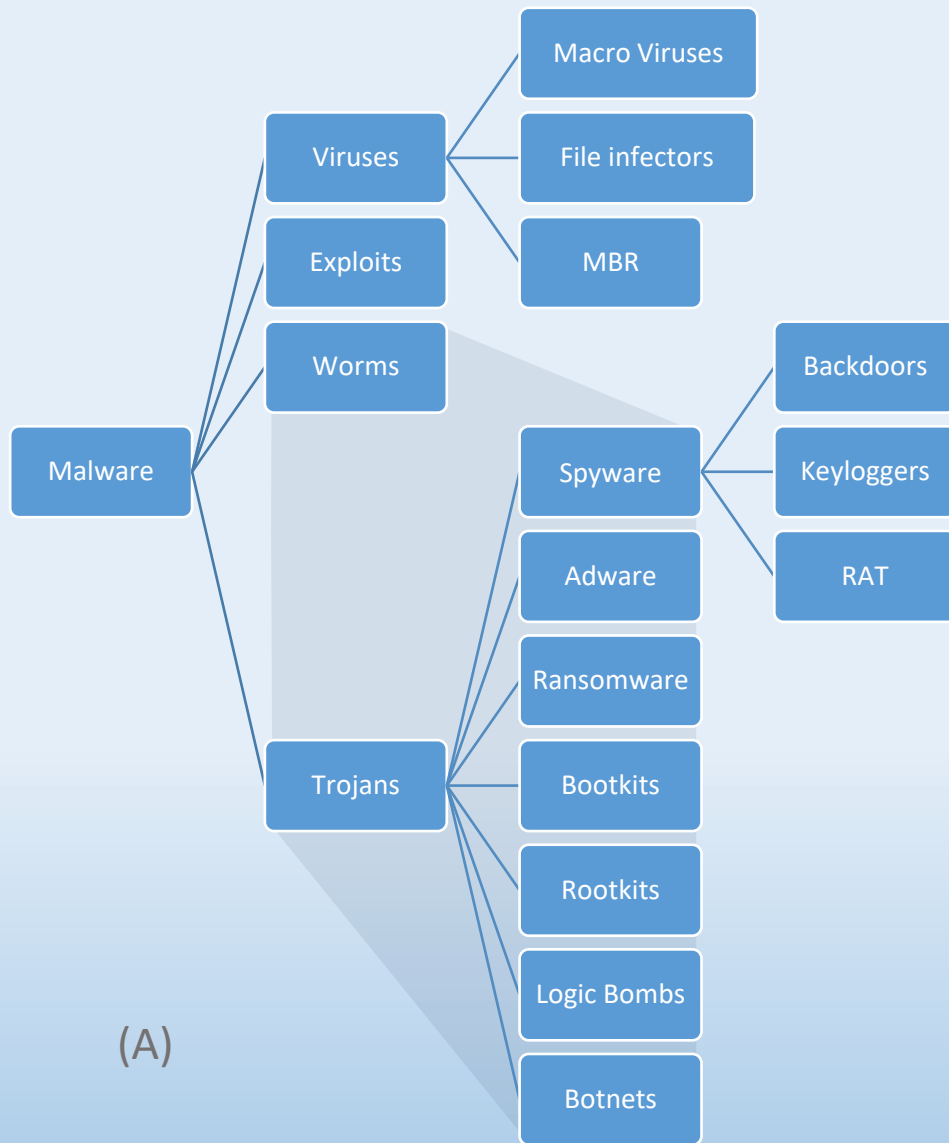
Descriere: Îndepărtarea intrărilor malware din registrul sistemului pentru a preveni executarea automată a malware-ului la pornirea sistemului.

Izolarea și Quarantina

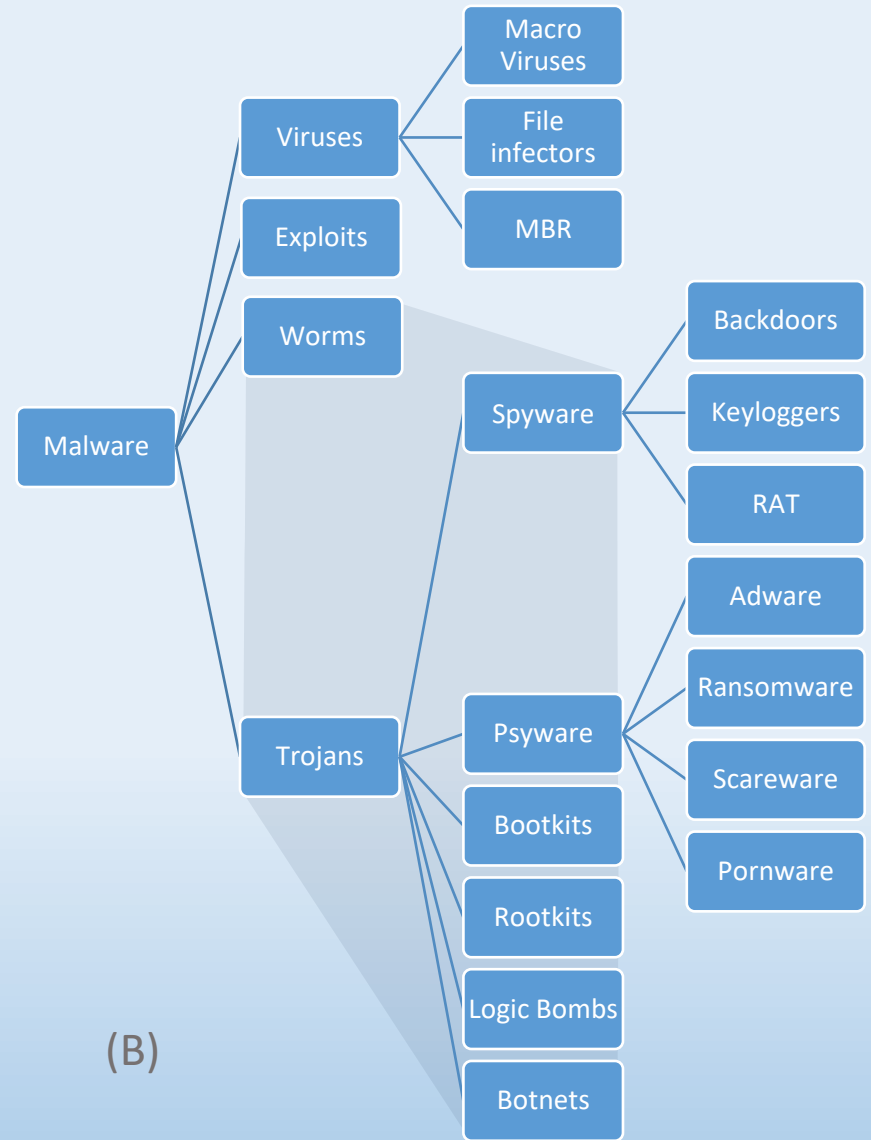
Descriere: Izolarea fișierelor suspecte sau infectate într-o zonă sigură (quarantină) pentru a preveni răspândirea malware-ului.

Schimbarea Parolelor

Descriere: Schimbarea parolelor conturilor afectate și a serviciilor online pentru a preveni accesul neautorizat.



(A)



(B)

non-polymorphic

Region Match

MD5 hash
signatures

db

```
Hash      : 345853ea3c6d7d5947be300d3cfe2e5b
Hex (I)   : E8 23 A9 42 C1 F0 10
Hex (II)  : E8 23 ?? ?? C1 F0 ?? 77
```

Chunks Match

Hexadecimal
signatures

db

Semnături vs cod polimorf

Frequency Match

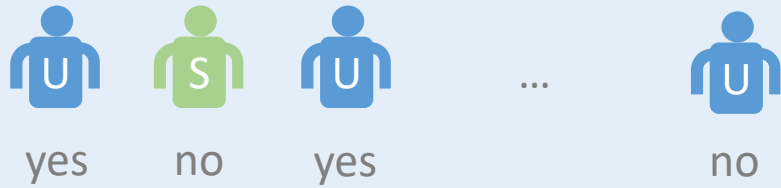
Heuristic
signatures

db

polymorphic

Catalogarea după părerea utilizatorilor:

Is this malware?



$$f(x) = \sum_{i=1}^m (weight(u_i) \times vote(x))$$

File x



$x1, x2, x3, \dots xn$

(A)

Is this IP/link malicious?



$$f(x) = \sum_{i=1}^m (weight(u_i) \times vote(x))$$

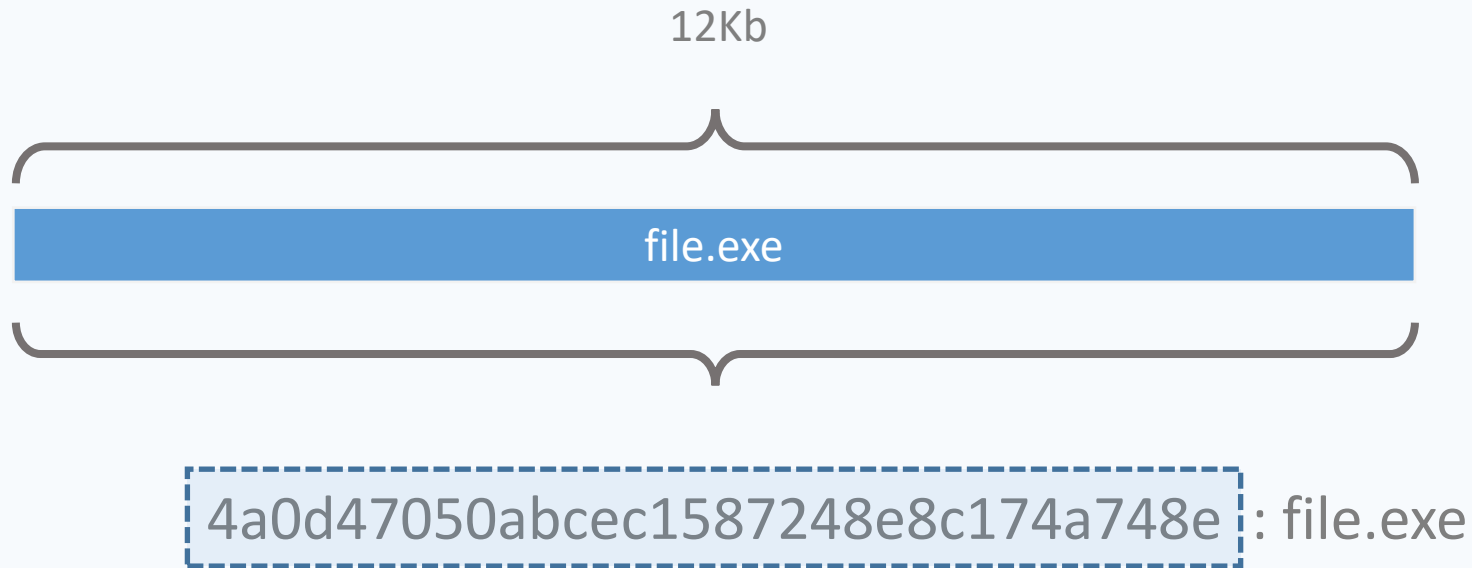
IP



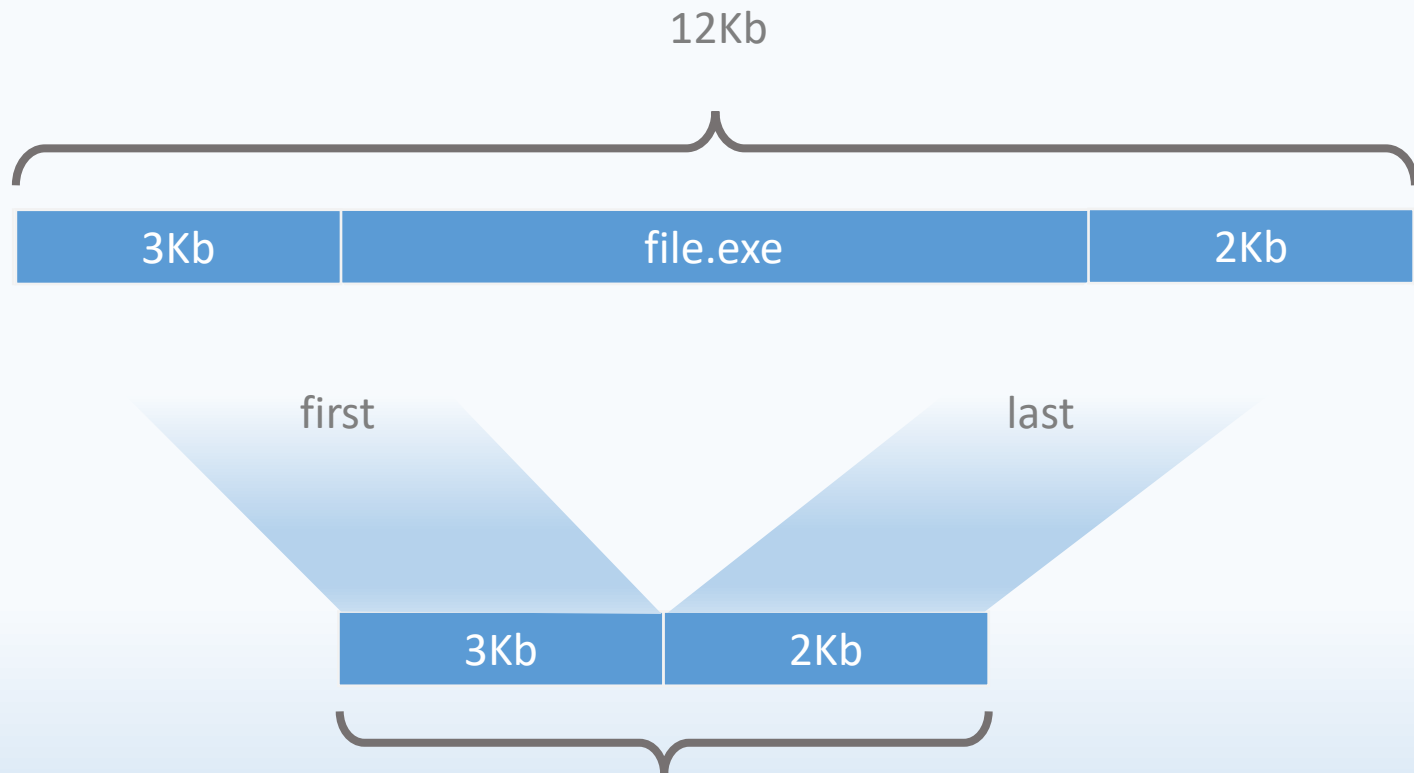
$x1, x2, x3, \dots xn$

(B)

MD5 signature

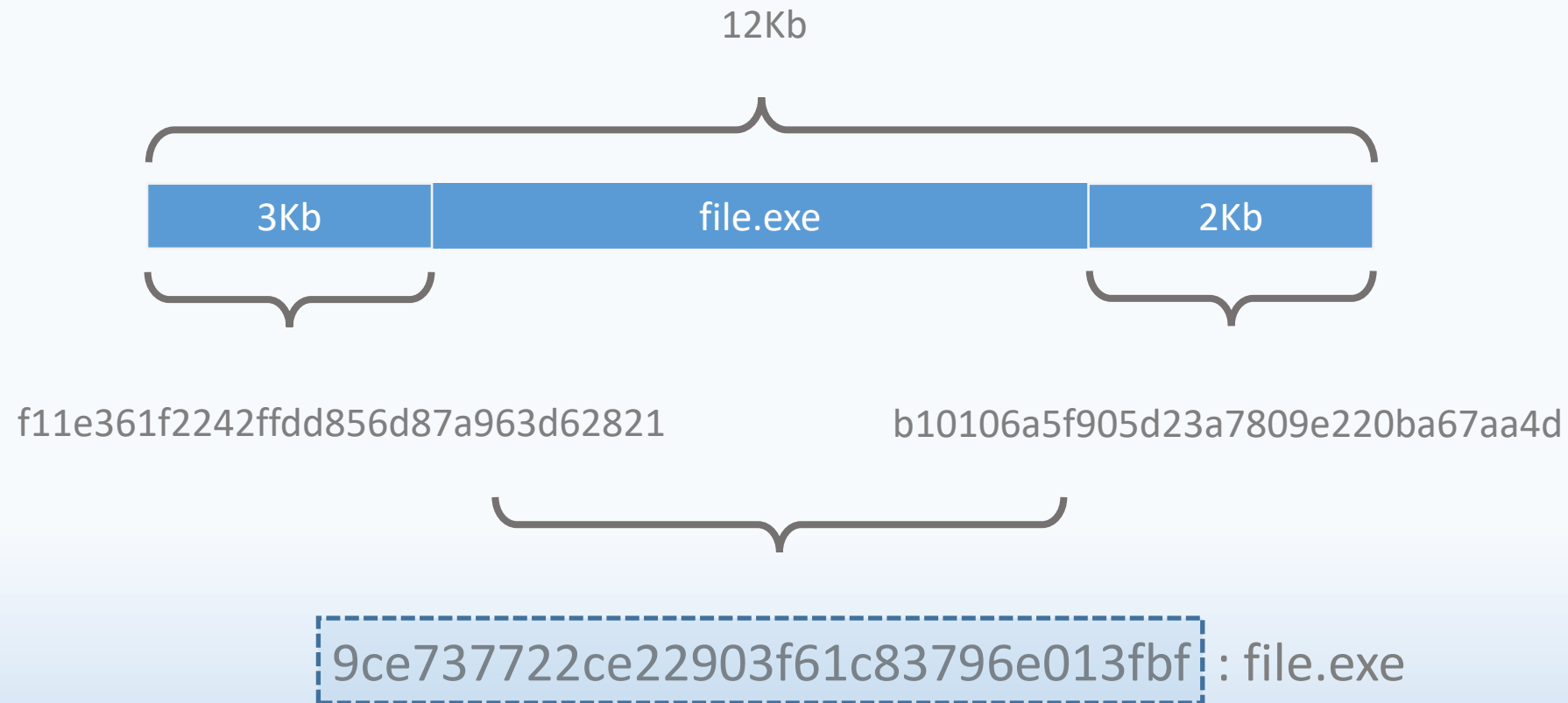


MD5 signature

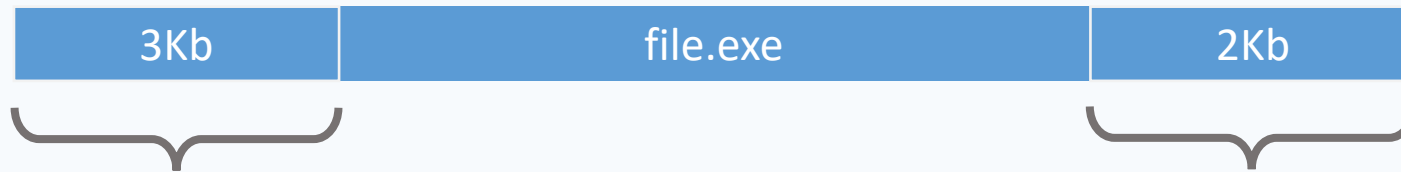


931ea08b1f3c1df63413b128f4c46f66 : file.exe

MD5 signature



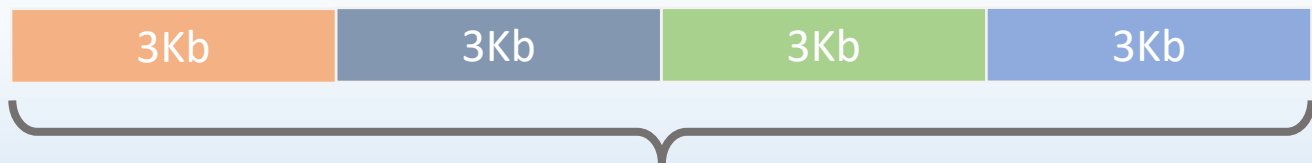
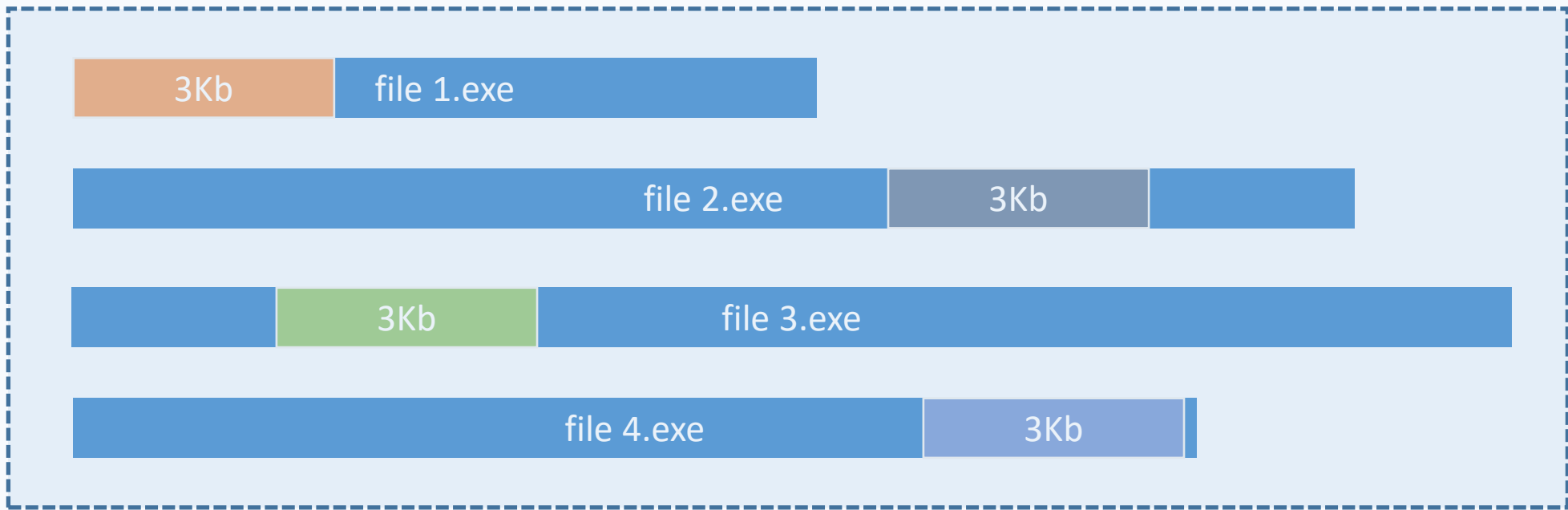
MD5 signature



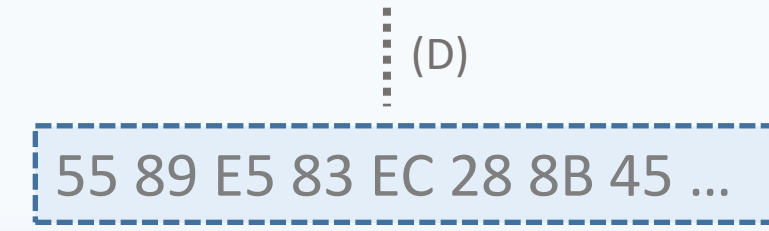
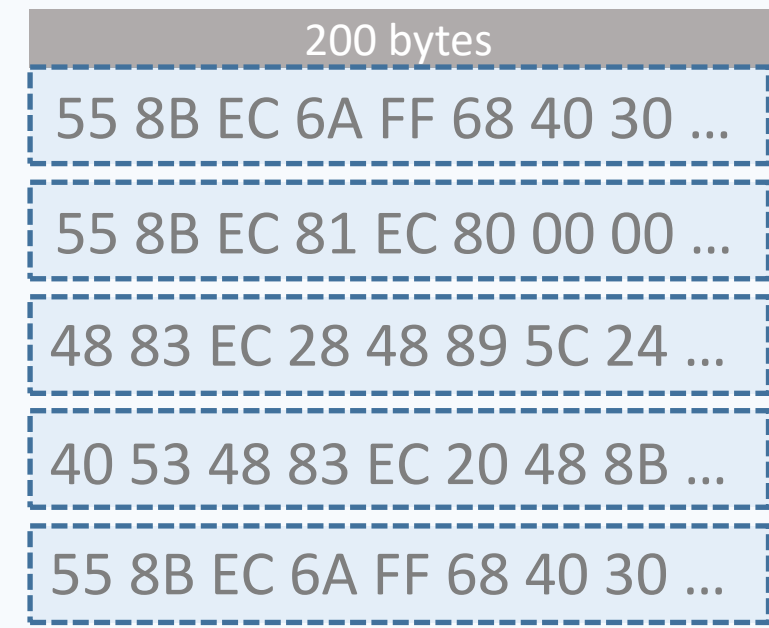
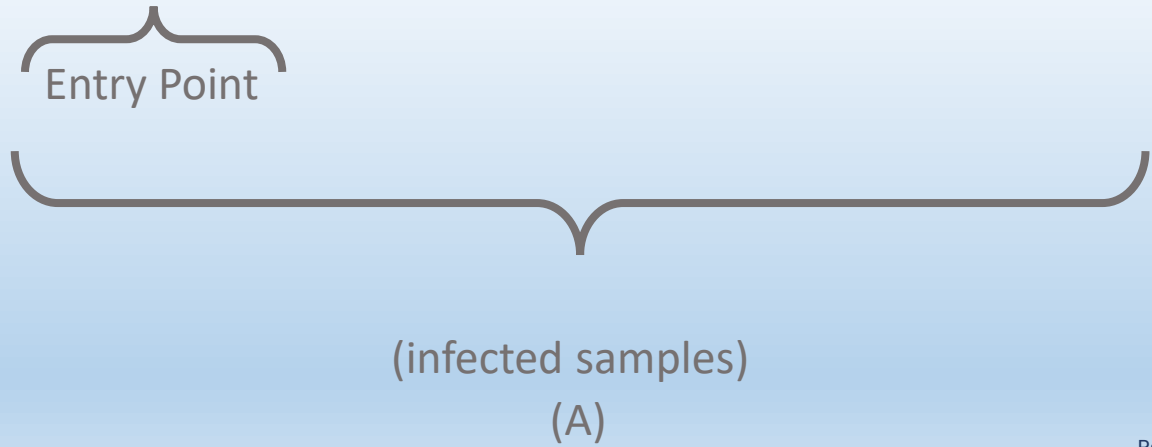
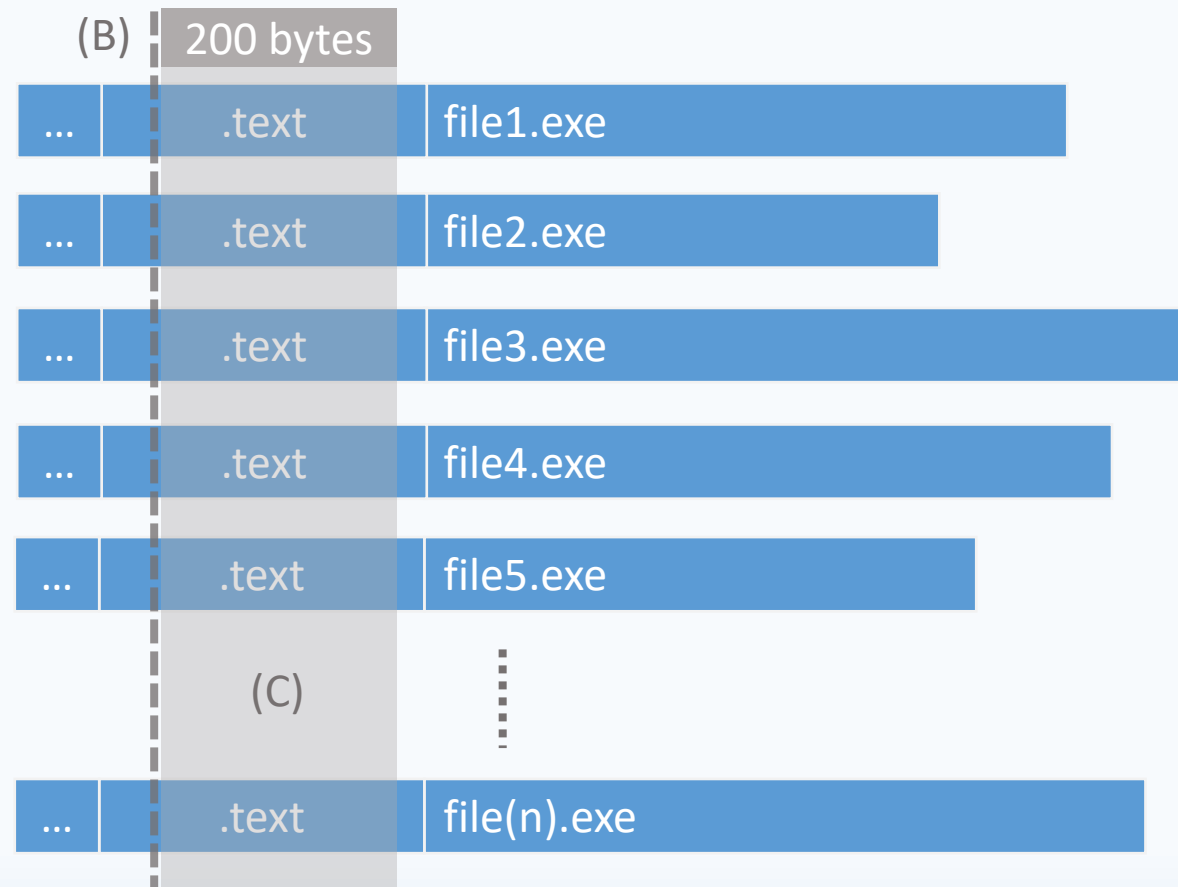
931ea08b1f3c1df63413b128f4c46f66

9ce737722ce22903f61c83796e013fbf

`7fa24b9db9b313523977d14b27a7a676` : `9ce737722ce22903f61c83796e013fbf` : file.exe



aa06d4dd7af1a739ecdce4630a2f7ed9



(F) PWM (pwm.json)

	1	2	3	...	199	200
00	0	0	0	...	0	0
01	0	0	0	...	10	4
...	...	...	...	...	...	...
FE	0	0	0	...	0	0
FF	0	0	0	...	1	12

(E)

PWM

(E)

Score

(D)

	1	2	3	...	199	200
00	0	0	0	...	0	0
01	0	0	0	...	10	4
...	...	...	...	...	...	...
FE	0	0	0	...	0	0
FF	0	0	0	...	1	12

= 1507

200 bytes

(C)

55 8B EC 6A FF 68 40 30 64 00 68 CC 04 60 00 64 ...

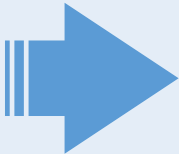
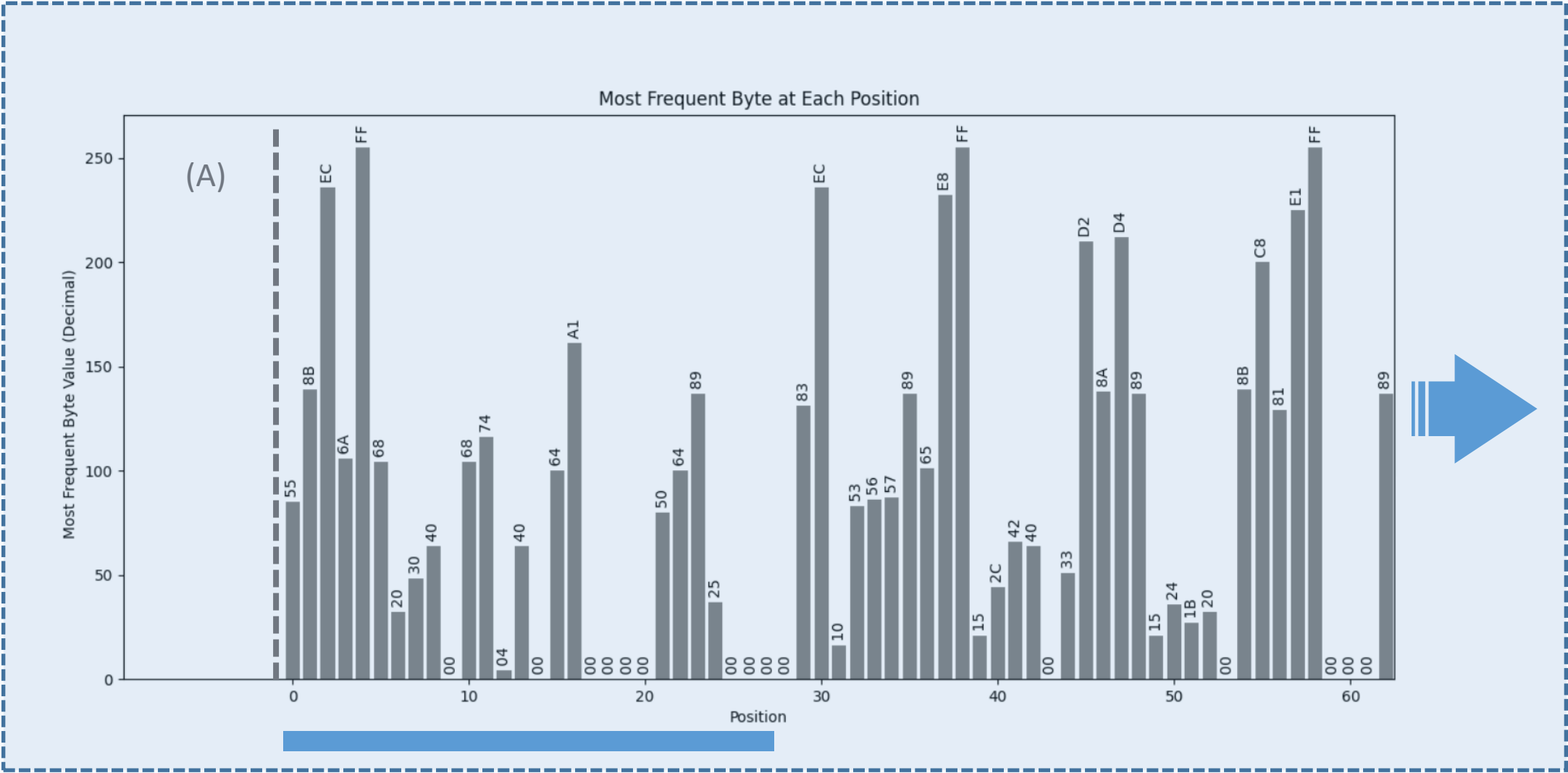
200 bytes (B)



(A)

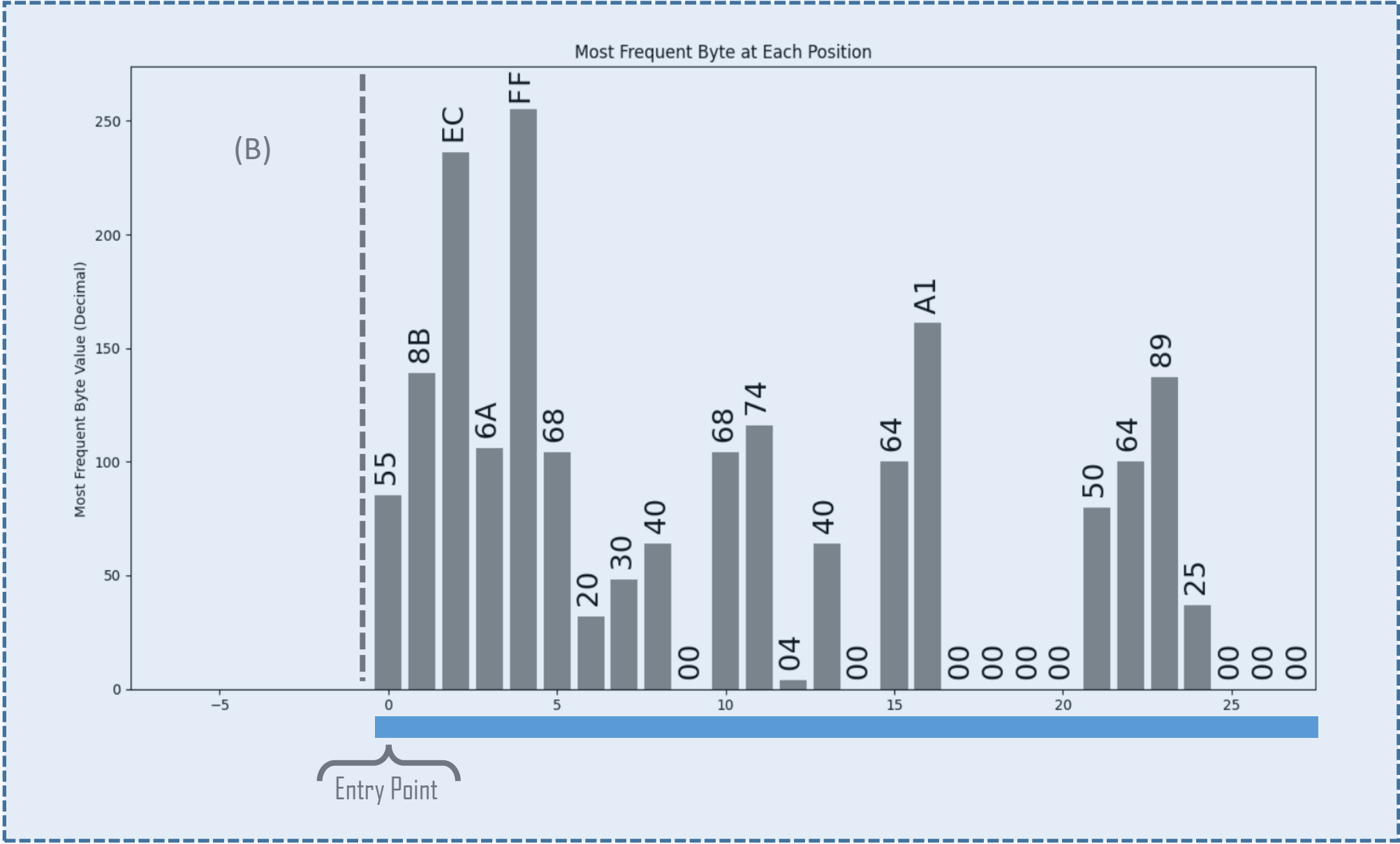
Entry Point

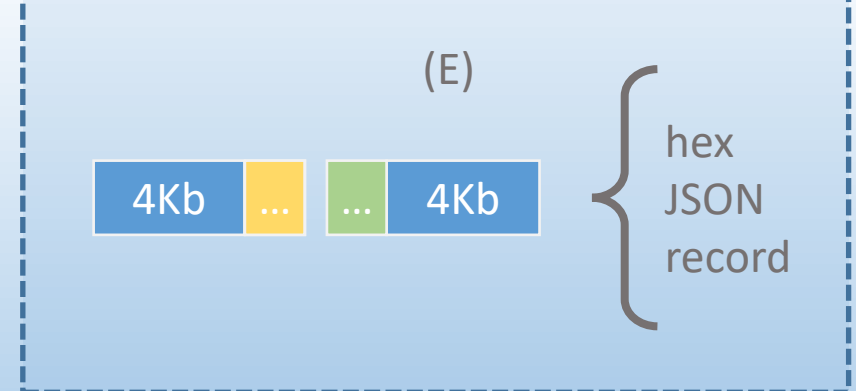
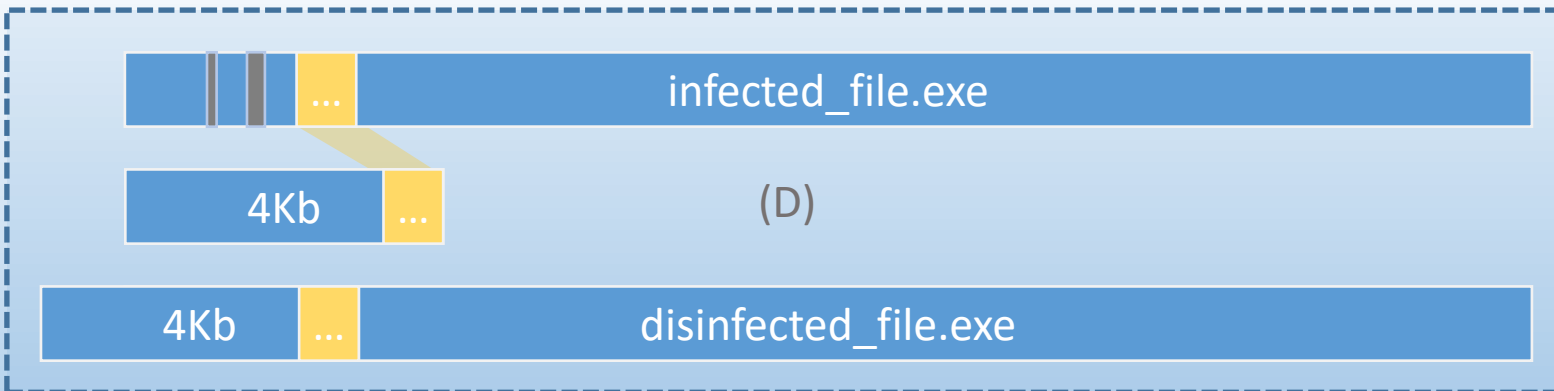
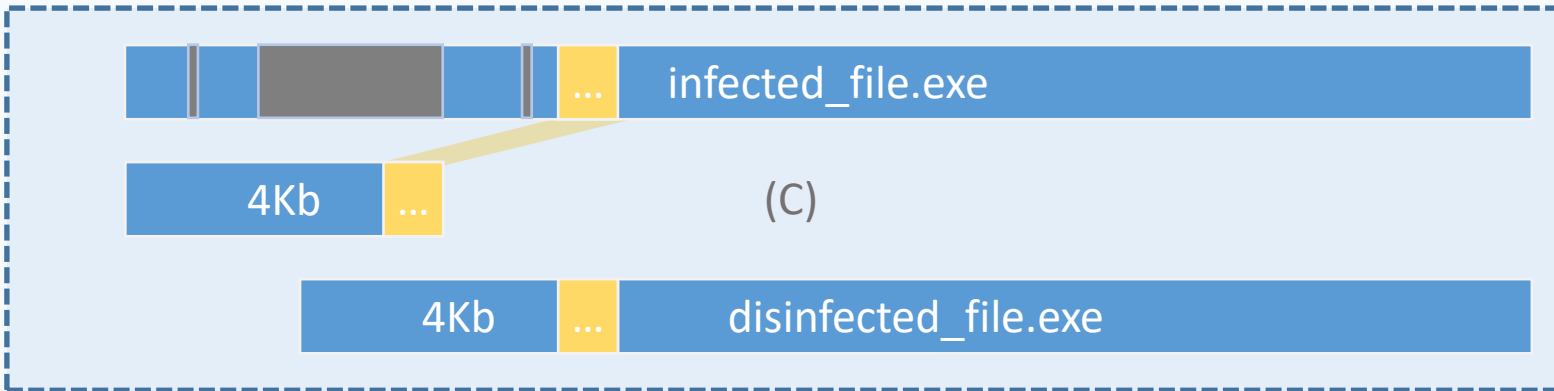
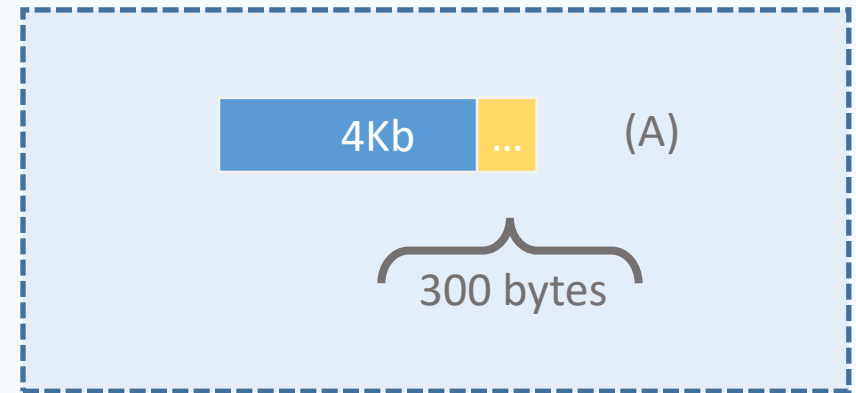
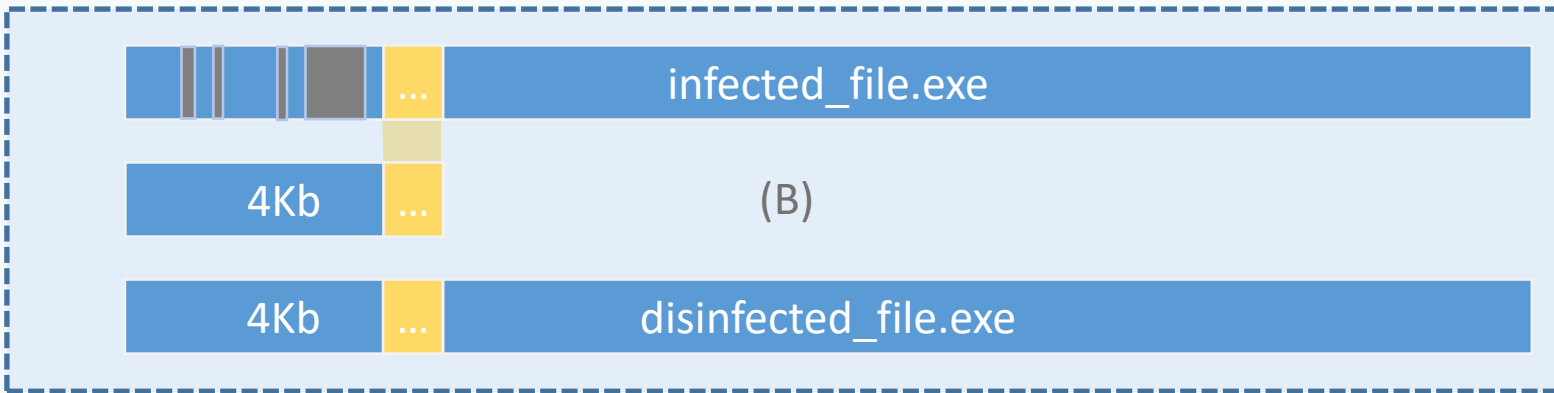
(file to be scanned)

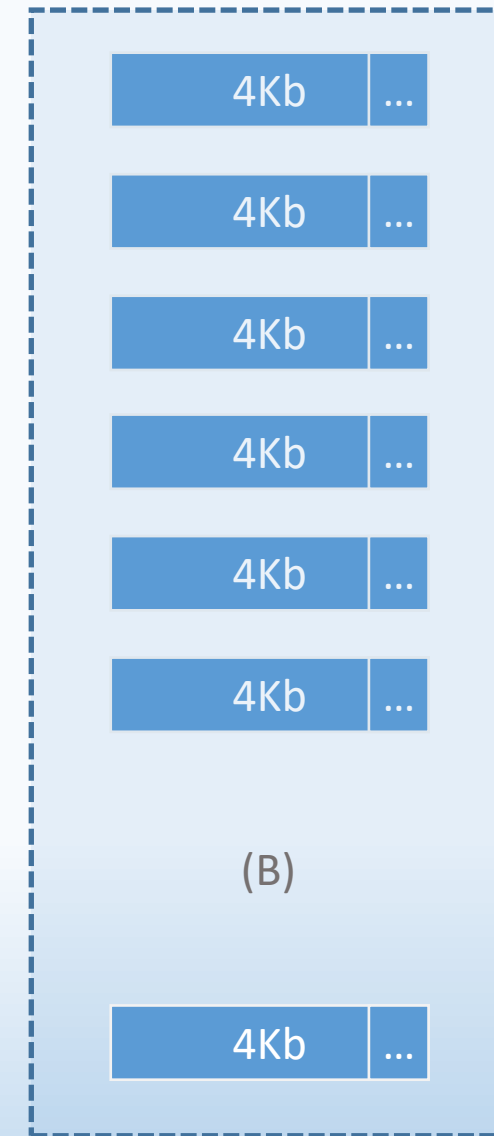
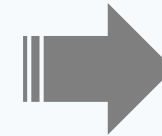
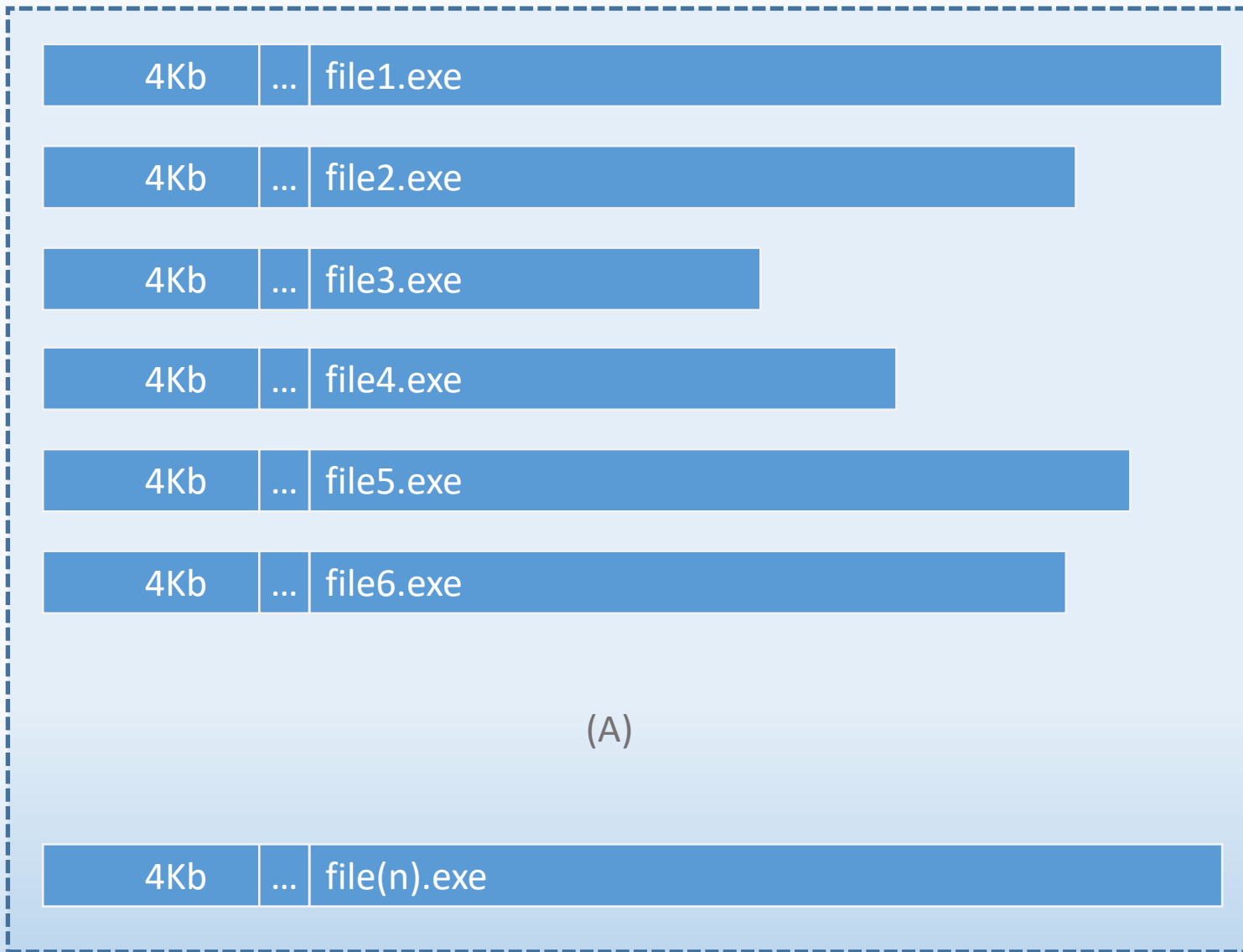


Entry Point



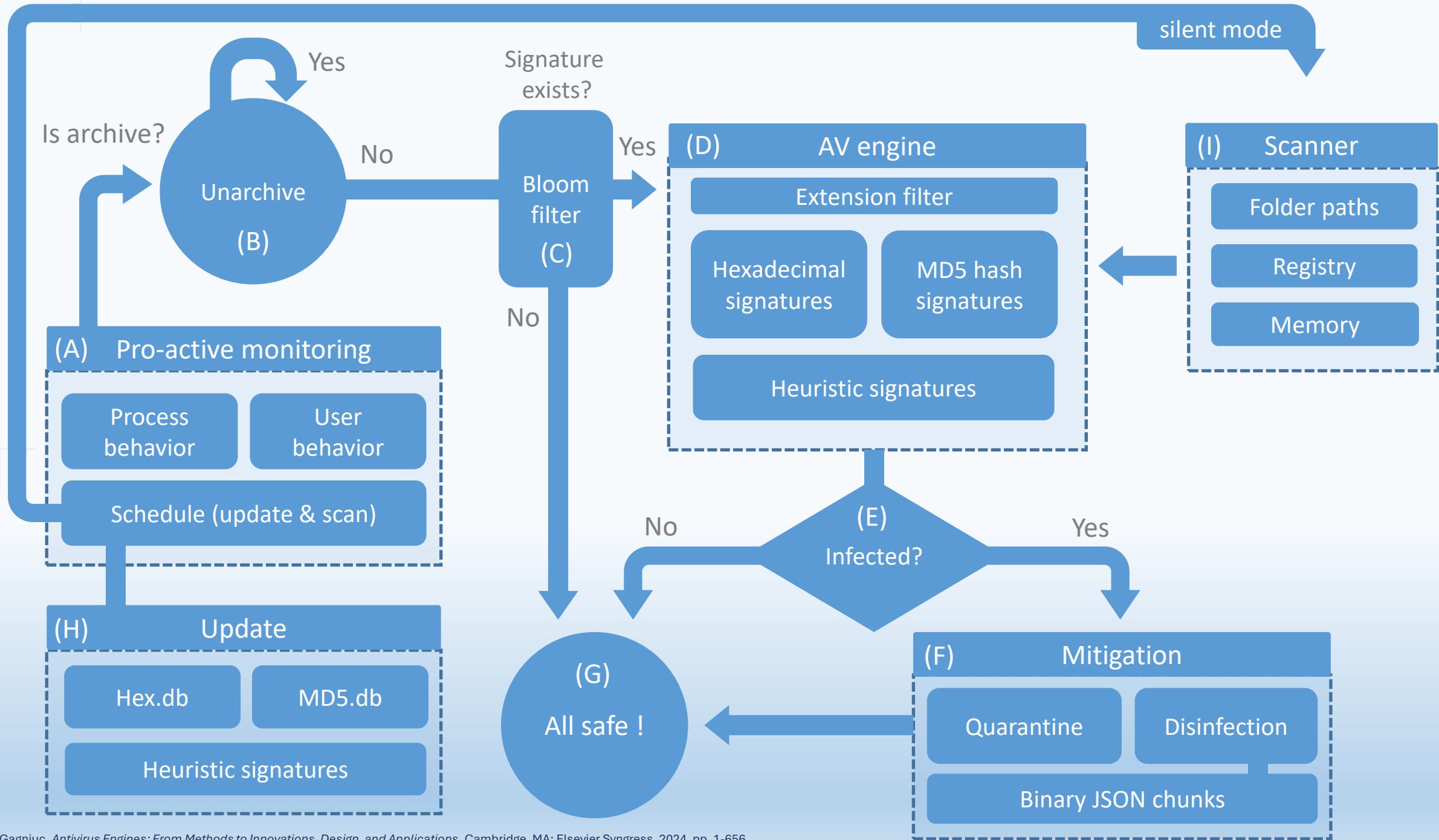






Original executable files

JSON



Temporary folder

file.zip

(F)

(C)

(A)

(D)

(B)

Delete

(E)

Malware scanner,
[if .exe]

Clean

File
send
to

Unpacker,
[if .zip]

[quarantine .zip]

file1_js7hd.exe

file2_4fjiv.exe

file3_yus5d.exe

file1_iry0u.zip

file3_pr4ew.exe

file1.exe

file1.txt

file1.pdf

file2.exe

file3.exe

file1.doc

file1.zip

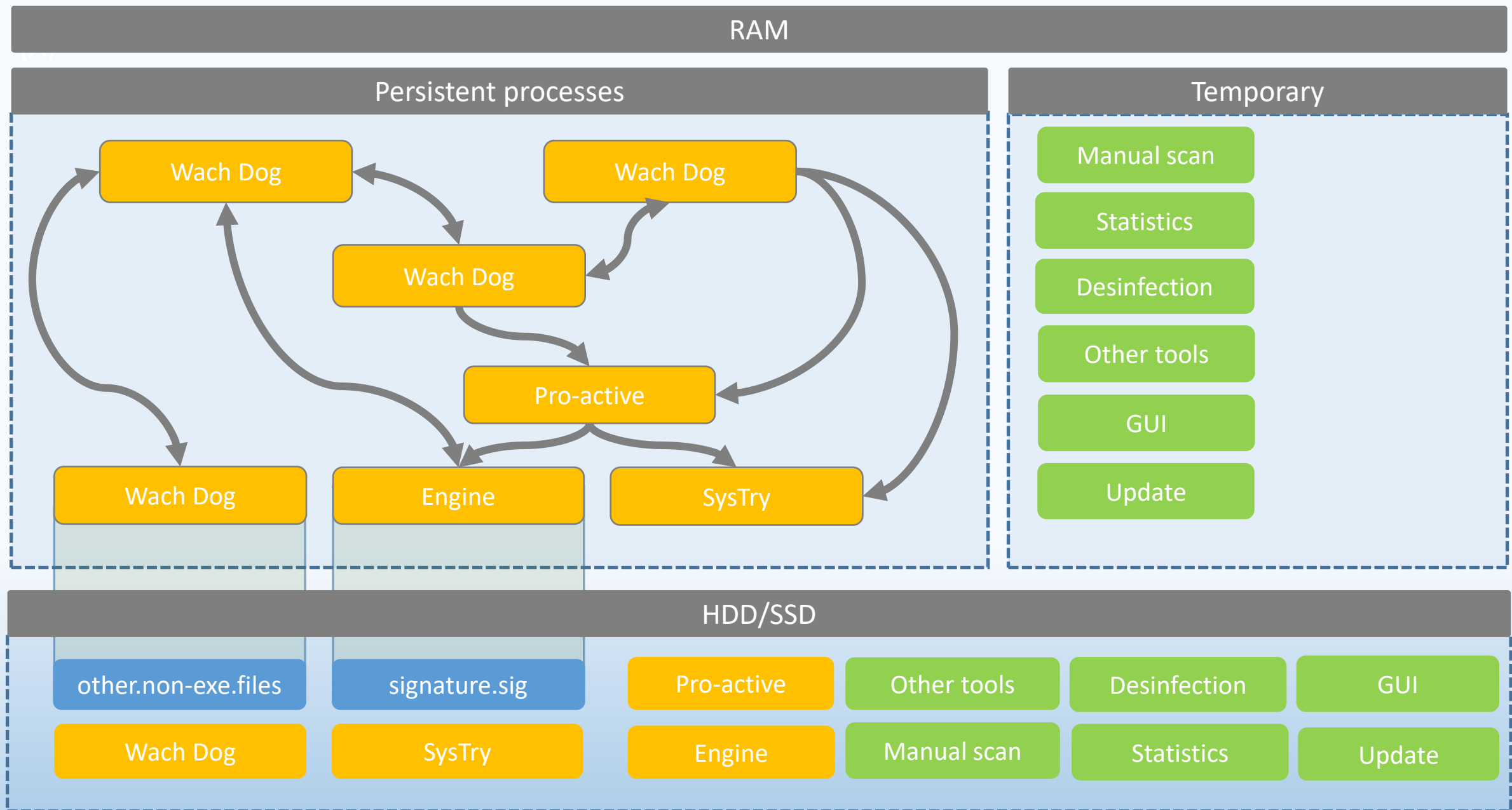
file1.xlsx

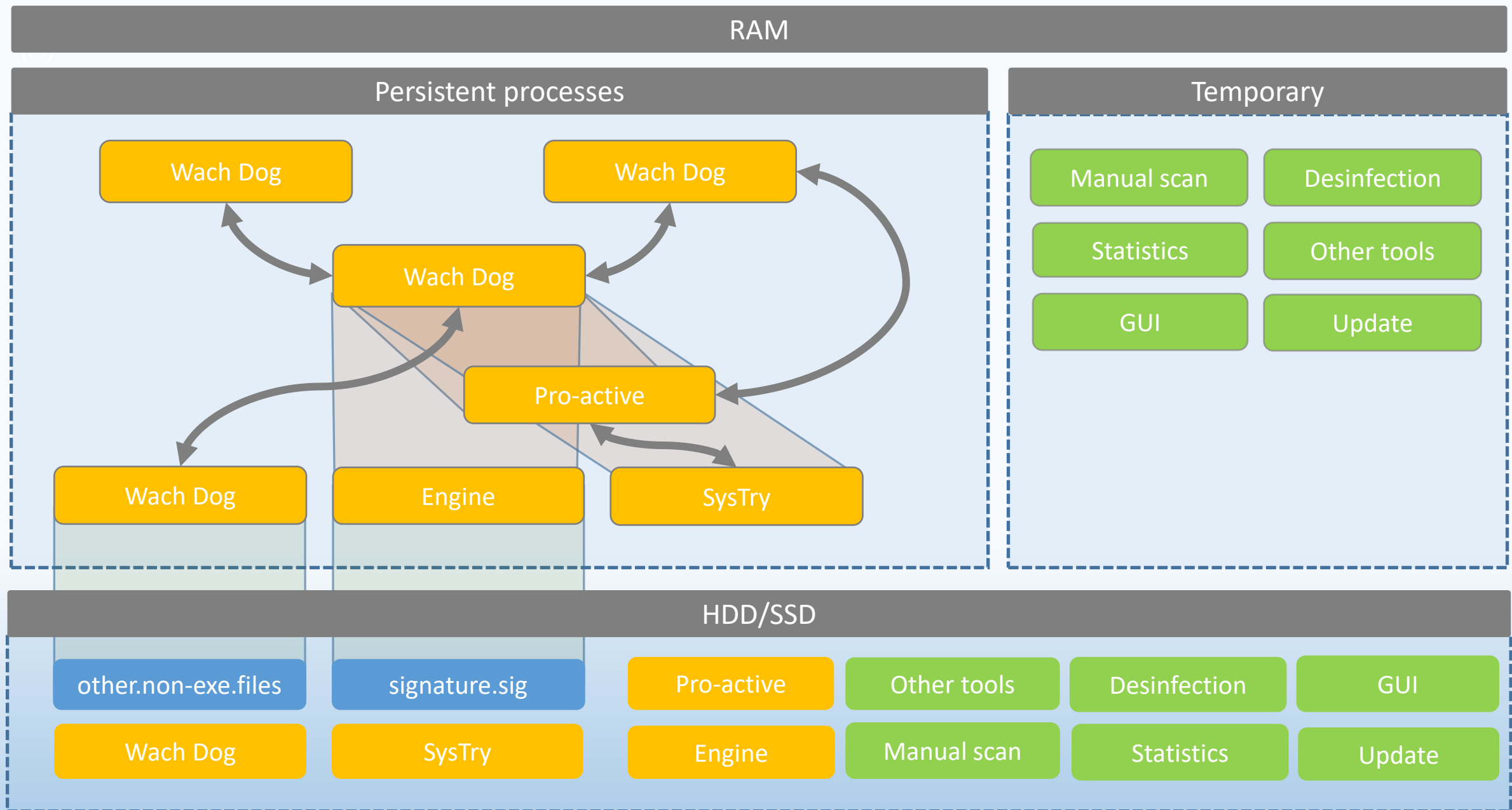
file3.exe

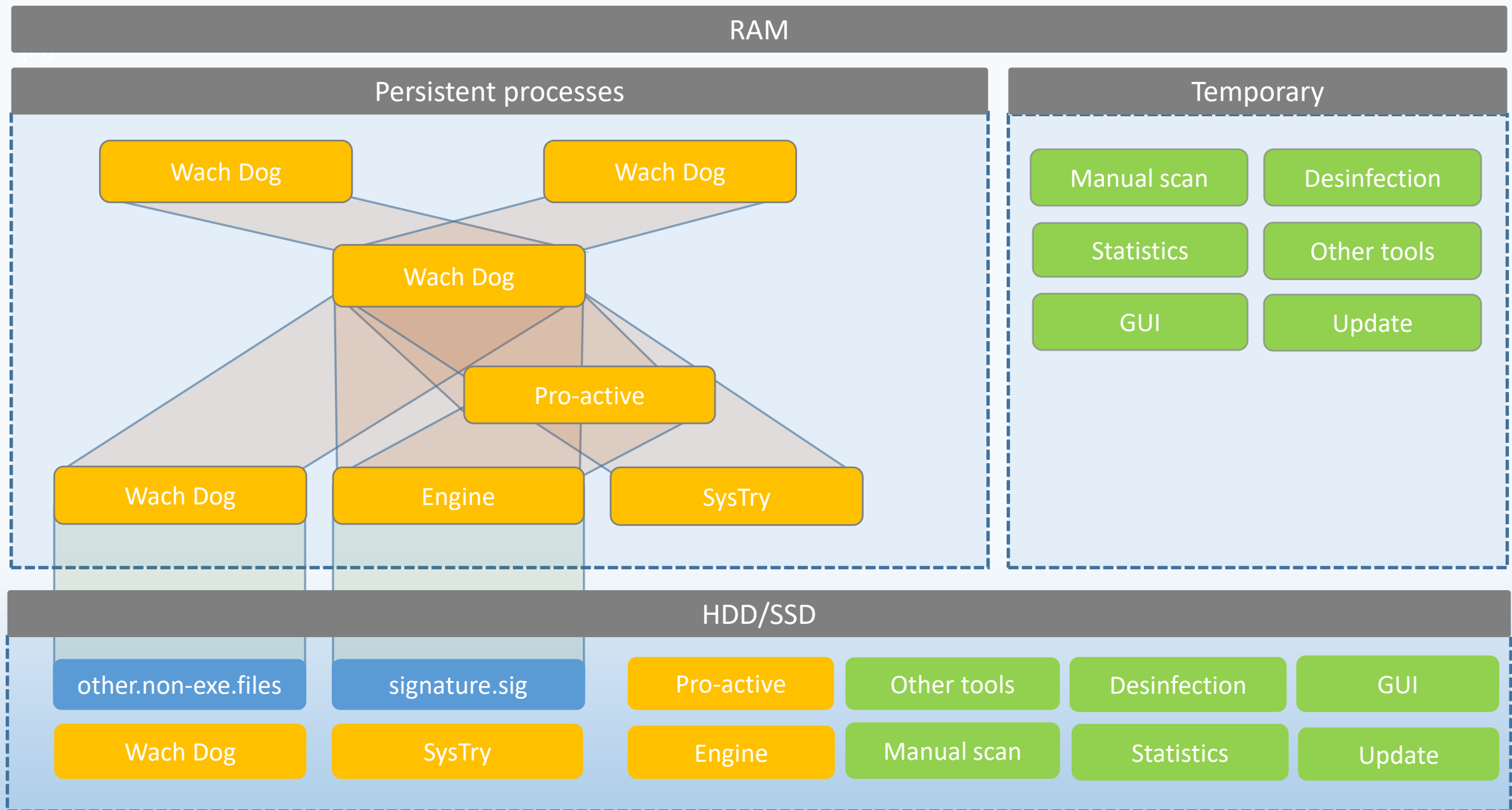
file1.sys

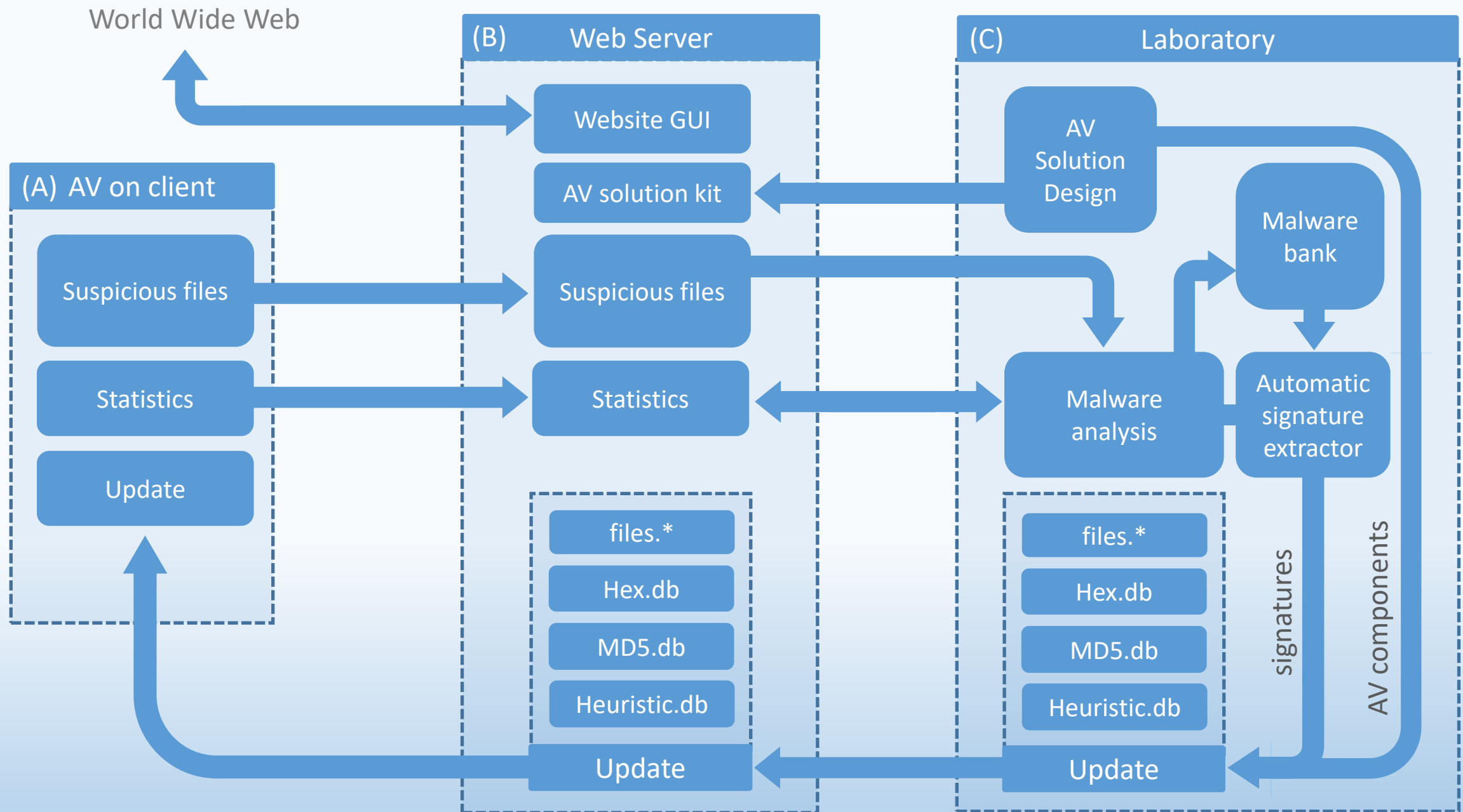
file2.doc

file2.txt









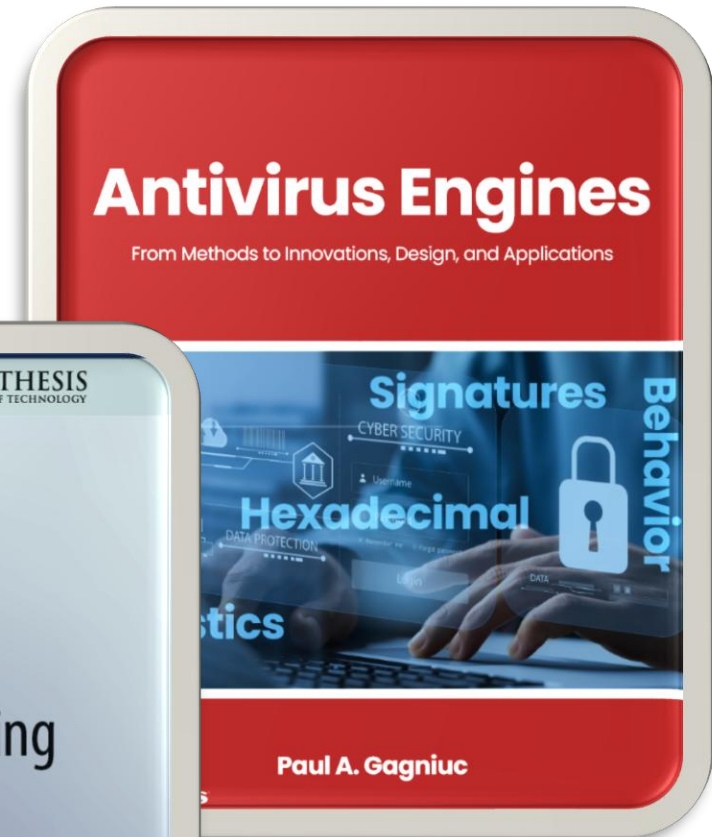
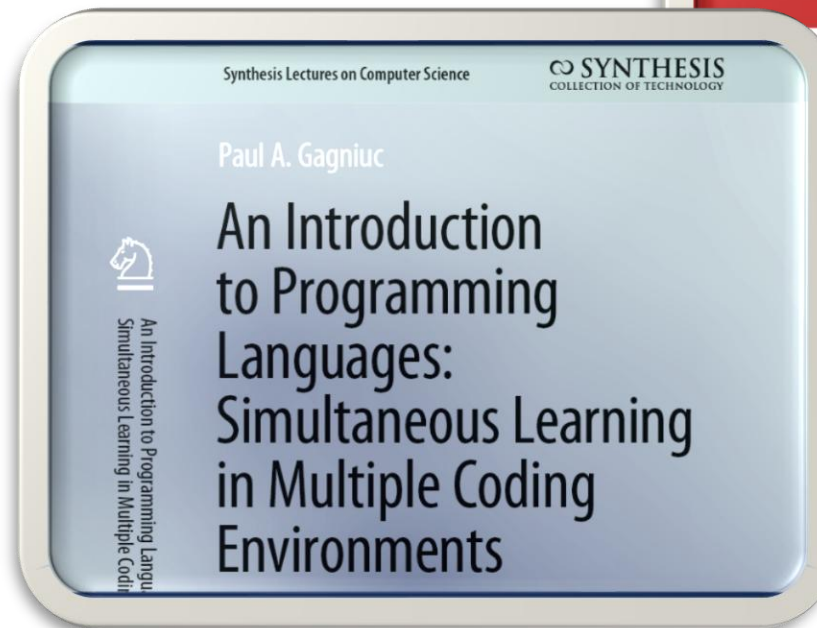
C.14.4

ALGORITMI FOLOSITI IN SECURITATEA CIBERNETICA



Algoritmi importanti in securitatea cibernetica cu aplicare directa in motoare de detectie malware:

- Algoritmi de alinieri de secvente
- Aliniere Globala (algoritmul Needleman-Wunsch)
- Aliniere Locala (algoritmul Smith–Waterman)
- Binary Search
- Hash tables
- Aho-Corasick
- Boyer-Moore string-search
- Filtrul Bloom
- Algoritmul de auto-aliniere



Hash tables

Tabelul de hash. Codul furnizat implementează o structură de date de bază de tabel hash în Python folosind tehnica de înlănțuire pentru rezoluția coliziunilor. Constă dintr-o clasă HashTable care reprezintă tabelul hash și conține metode de inserare a perechilor cheie-valoare și de căutare a valorilor asociate cheilor. Clasa HashTable are un constructor care inițializează tabelul hash cu o dimensiune specificată. Acesta creează o listă goală de liste (găleți) pentru a stoca perechile cheie-valoare. O metodă privată `_hash_function(self, key)` calculează valoarea hash pentru o anumită cheie folosind funcția încorporată Python `hash()` și operatorul modulo pentru a se potrivi cu valoarea în dimensiunea tabelului hash. Metoda `insert(self, key, value)` adaugă o pereche cheie-valoare în tabelul hash. Acesta calculează indexul compartimentului folosind `_hash_function()` și adaugă perechea la compartimentul corespunzător. Metoda `search(self, key)` caută o valoare asociată cu o anumită cheie în tabelul hash. Acesta calculează indexul compartimentului folosind `_hash_function()` și iterează prin perechile din compartiment pentru a găsi cheia potrivită. Dacă este găsit, returnează valoarea asociată; în caz contrar, returnează Niciunul. În practică, o instanță a clasei HashTable este creată și perechile cheie-valoare sunt inserate folosind metoda `insert()`. Metoda `search()` este apoi utilizată pentru a prelua valori bazate pe chei. Codul demonstrează principiile de bază ale unui tabel hash, inclusiv stocarea și preluarea eficientă a perechilor cheie-valoare prin funcția hash și tehnica de înlănțuire pentru gestionarea coliziunilor hash.



```
class HashTable:
    def __init__(self, size):
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash_function(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self._hash_function(key)
        self.table[index].append((key, value))

    def search(self, key):
        index = self._hash_function(key)
        for stored_key, value in self.table[index]:
            if stored_key == key:
                return value
        return None

# create a hash table and insert some key-value pairs:
hash_table = HashTable(size=10)
hash_table.insert("d41d8cd98f00b204e9800998ecf8427e", "Trojan")
hash_table.insert("098f6bcd4621d373cade4e832627b4f6", "Rootkit")
hash_table.insert("1f0e3dad99908345f7439f8ffabdfc4", "Bootkit")

# search for values using keys:
print(hash_table.search("d41d8cd98f00b204e9800998ecf8427e"))
print(hash_table.search("098f6bcd4621d373cade4e832627b4f6"))
print(hash_table.search("1f0e3dad99908345f7439f8ffabdfc4"))
print(hash_table.search("c4ca4238a0b923820dcc509a6f75849b"))
```

Output:

```
Trojan
Rootkit
Bootkit
None
```

Aho-Corasick

Algoritmul Aho-Corasick. În acest exemplu, clasa AhoCorasickNode reprezintă un nod în încercarea Aho-Corasick. Clasa AhoCorasick construiește trie și se ocupă de construcția tranzițiilor de eșec. Metoda este folosită pentru a căuta modele în textul dat. Codul furnizat de mai sus implementează algoritmul Aho-Corasick, o tehnică rapidă și eficientă de potrivire a șirurilor folosită pentru a identifica mai multe modele predefinite (cuvinte cheie) într-un text dat. Algoritmul este util în special pentru sarcinile care implică căutarea mai multor cuvinte cheie în corpuri mari de text. Codul este format din două clase principale. a) Clasa AhoCorasickNode reprezintă noduri în structura trie utilizată de algoritm. Fiecare nod conține informații despre tranzițiile către alte noduri, modele de ieșire asociate nodului și un indicator de stare de eșec care ajută la traversare. b) Clasa AhoCorasick este responsabilă de implementarea algoritmului. Include următoarele metode: i) Metoda `__init__(self, patterns)` inițializează algoritmul prin construirea trie-ului folosind o listă de modele de intrare. ii) Metoda `build_trie(self, patterns)` construiește structura trie prin adăugarea iterativă a fiecărui model caracter cu caracter. iii) Metoda `build_failures(self)` construiește tranziții de eșec pentru noduri, creând legături către stările de eșec corespunzătoare. iv) Metoda `search(self, text)` caută modele în textul de intrare folosind trie construit și returnează o listă de modele potrivite. În exemplul de utilizare furnizat, este definită o listă de modele ["sig1", "sig2", "sig3", "sig4"], iar textul „IASig3mAFile” este folosit pentru testare. O instanță a clasei AhoCorasick este creată cu modelele, iar metoda de căutare este invocată pentru a găsi modele în text. Modelele potrivite sunt apoi imprimare.



```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.output = []

    def build_trie(patterns):
        root = TrieNode()
        for pattern in patterns:
            node = root
            for char in pattern:
                if char not in node.children:
                    node.children[char] = TrieNode()
                node = node.children[char]
            node.output.append(pattern)
        return root

    def aho_corasick(text, root):
        results = []
        node = root
        for i, char in enumerate(text):
            while char not in node.children and node != root:
                node = node.fail
            if char in node.children:
                node = node.children[char]
            for pattern in node.output:
                results.append((i - len(pattern) + 1, pattern))
        return results

    def build_fail_links(root):
        queue = []
        for node in root.children.values():
            node.fail = root
            queue.append(node)
        while queue:
            current = queue.pop(0)
            for char, child in current.children.items():
                queue.append(child)
                fail_state = current.fail
                while char not in fail_state.children and fail_state != root:
                    fail_state = fail_state.fail
                child.fail = fail_state.children.get(char, root)

if __name__ == "__main__":
    patterns = ["sig1", "sig2", "sig3", "sig4"]
    text = "IASig3mAFile"

    root = build_trie(patterns)
    build_fail_links(root)
    results = aho_corasick(text, root)

    if results:
        print("Found patterns:")
        for position, pattern in results:
            print(f"Pattern '{pattern}' found at position {position}")
    else:
        print("No patterns found.")
```

Output:

```
Found patterns:
Pattern 'sig3' found at position 2
```

Filtrul Bloom

Codul din dreapta definește o clasă Python numită BloomFilter care implementează o structură de date de bază a filtrului Bloom. Un filtru Bloom este o structură de date probabilistică utilizată pentru testarea apartenenței, în special pentru a verifica eficient dacă un element este sau nu membru al unui set. Clasa BloomFilter este inițializată cu doi parametri: dimensiune și hash_functions. size determină dimensiunea (numărul de biți) a matricei de biți de filtru Bloom, reprezentând capacitatea sa de a stoca elemente. hash_functions este o listă de funcții hash utilizate de filtru pentru hasharea elementelor. Mai multe funcții hash sunt folosite pentru a distribui elementele în matricea de biți. Metoda constructorului (__init__) inițializează filtrul Bloom setându-i dimensiunea, creând o matrice de biți (inițializată cu zerouri) și stocând funcțiile hash furnizate. Metoda de adăugare vă permite să adăugați elemente la filtrul Bloom. Pentru fiecare funcție hash din lista hash_functions, calculează un index în matricea de biți prin aplicarea funcției hash la element și luând modulo rezultat cu dimensiunea filtrului. Apoi setează bitul corespunzător din matricea de biți la 1, indicând faptul că acest element este prezent (sau ar putea fi prezent) în filtru. Metoda __contains__ supraîncărcă operatorul in, permițând cuiva să verifice dacă un element este prezent în filtru. Pentru fiecare funcție hash din lista hash_functions, calculează un index în matricea de biți în același mod ca metoda add. Dacă oricare dintre biții corespunzători din matricea de biți este 0, returnează imediat False, indicând că elementul cu siguranță nu este prezent în filtru. Dacă toți biții corespunzători sunt 1, returnează True, indicând faptul că elementul ar putea fi prezent în filtru. Cu toate acestea, sunt posibile rezultate false pozitive. Exemplul de utilizare demonstrează cum să creați un BloomFilter cu o dimensiune de 100 și două funcții hash (funcția hash încorporată).



```
class BloomFilter:
    def __init__(self, size, hash_functions):
        self.size = size
        self.bit_array = [0] * size
        self.hash_functions = hash_functions

    def add(self, item):
        for fn in self.hash_functions:
            index = fn(item) % self.size
            self.bit_array[index] = 1

    def __contains__(self, item):
        for fn in self.hash_functions:
            index = fn(item) % self.size
            if self.bit_array[index] == 0:
                return False
        return True

# create a Bloom
# filter with a size
# of 100 and two
# hash functions:
bloom_filter = BloomFilter(100, [hash, hash])

# add items to the filter:
bloom_filter.add("sig1")
bloom_filter.add("sig2")

# are the items in
# the filter already?:
print("Is 'sig1' in the filter?", "sig1" in bloom_filter)
print("Is 'sig3' in the filter?", "sig3" in bloom_filter)
```

Output:

```
Is 'sig1' in the filter? True
Is 'sig3' in the filter? False
```



Antivirus Engines

From Methods to Innovations, Design, and Applications

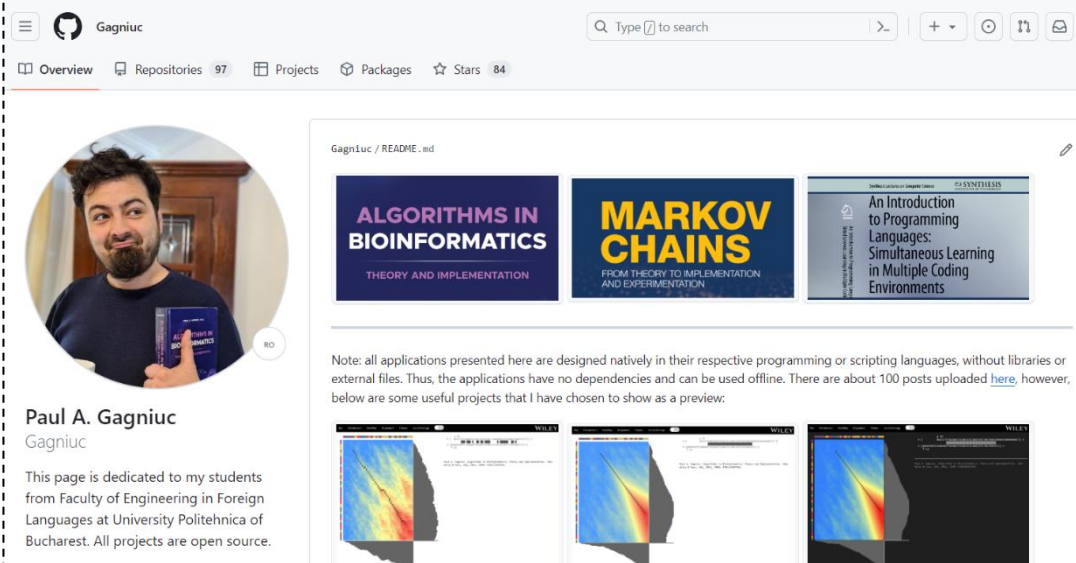
Paul A. Gagniuc. *Antivirus Engines: From Methods to Innovations, Design, and Applications*. Cambridge, MA: Elsevier Syngress, 2024. pp. 1-656.



BIBLIOGRAFIE / RESURSE

- Paul A. Gagniuc. *Antivirus Engines: From Methods to Innovations, Design, and Applications*. Cambridge, MA: Elsevier Syngress, 2024. pp. 1-656.
- Paul A. Gagniuc. *An Introduction to Programming Languages: Simultaneous Learning in Multiple Coding Environments. Synthesis Lectures on Computer Science*. Springer International Publishing, 2023, pp. 1-280.
- Paul A. Gagniuc. *Coding Examples from Simple to Complex - Applications in MATLAB*, Springer, 2024, pp. 1-255.
- Paul A. Gagniuc. *Coding Examples from Simple to Complex - Applications in Python*, Springer, 2024, pp. 1-245.
- Paul A. Gagniuc. *Coding Examples from Simple to Complex - Applications in Javascript*, Springer, 2024, pp. 1-240.
- Paul A. Gagniuc. *Markov chains: from theory to implementation and experimentation*. Hoboken, NJ, John Wiley & Sons, USA, 2017, ISBN: 978-1-119-38755-8.

<https://github.com/gagniuc>



The screenshot shows the GitHub profile of Gagniuc. The header includes the username 'Gagniuc', a search bar, and navigation links for Overview, Repositories (97), Projects, Packages, Stars (84), and a mail icon. The profile picture shows a man with a beard holding a book. Below the profile picture, the name 'Paul A. Gagniuc' and the bio 'This page is dedicated to my students from Faculty of Engineering in Foreign Languages at University Politehnica of Bucharest. All projects are open source.' are visible. The main content area displays a 'README.md' file with three book covers: 'ALGORITHMS IN BIOINFORMATICS', 'MARKOV CHAINS', and 'An Introduction to Programming Languages: Simultaneous Learning in Multiple Coding Environments'. A note below the covers states: 'Note: all applications presented here are designed natively in their respective programming or scripting languages, without libraries or external files. Thus, the applications have no dependencies and can be used offline. There are about 100 posts uploaded [here](#), however, below are some useful projects that I have chosen to show as a preview.' At the bottom, three small images of heatmaps or data visualizations are shown.